

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО

Навчально-науковий інститут електричної інженерії та інформаційних
технологій

Кафедра комп'ютерної інженерії та електроніки

ПОЯСНЮВАЛЬНА ЗАПИСКА

до кваліфікаційної роботи бакалавра

на тему:

«AI-система семантичного пошуку в корпусі освітніх документів на
основі LLM з інтерактивною графовою навігацією результатів»

здобувача Грищенко Ілля Володимирович
спеціальності 123 – «Комп'ютерна інженерія»

ОПП «Комп'ютерна інженерія»

курсу IV групи КІ-22-1

Керівник: _____

Кременчук 2026

ЗАВДАННЯ НА КВАЛІФІКАЦІЙНУ РОБОТУ

Тема роботи: «AI-система семантичного пошуку в корпусі освітніх документів на основі LLM з інтерактивною графовою навігацією результатів». Вихідними даними є монорепозіторій EduGraph, звіт з переддипломної практики, корпус освітніх документів у Google Drive, а також методичні вимоги до структури пояснювальної записки.

Мета роботи полягає у розробленні та обґрунтуванні веб-системи, яка автоматизує синхронізацію освітніх документів, виконує семантичний пошук за змістом і подає результати в інтерактивній графовій формі. Робота має довести не тільки технічну працездатність прототипу, а й доцільність обраної архітектури для освітнього середовища.

- проаналізувати предметну область пошуку в освітніх документах і визначити обмеження традиційного пошуку за словами;
- розглянути типову клієнт-серверну архітектуру системи семантичного пошуку;
- сформулювати функціональні та нефункціональні вимоги до системи EduGraph;
- спроектувати ролі користувачів, сценарії роботи та взаємодію компонентів;
- описати модель даних, синхронізацію Google Drive, механізм індексації та vector search;
- розкрити реалізацію ASP.NET Core API, React-клієнта та FastAPI-сервісу;
- підготувати інструкцію користувача, рекомендації щодо експлуатації та графічні матеріали.

Примітка: цей аркуш є змістовним наповненням для подальшого перенесення у форму кафедри.

КАЛЕНДАРНИЙ ПЛАН

Етап	Зміст робіт	Результат
1	Аналіз предметної області, корпусу документів, ролей і проблем пошуку.	Сформована постановка задачі.
2	Вибір архітектури та технологій: ASP.NET Core, React, FastAPI, SQLite, LanceDB.	Обґрунтовано технологічну основу.
3	Проектування бази даних, endpoint-ів, сценаріїв синхронізації та пошуку.	Підготовлено проєктні рішення.
4	Реалізація backend API, авторизації, заявок, ролей і Google Drive інтеграції.	Створено серверну частину.
5	Реалізація Python-сервісу embeddings, chunking, LanceDB і reranking.	Створено сервіс семантичного пошуку.
6	Реалізація React SPA, маршрутизації, графу результатів і адміністративних екранів.	Створено клієнтську частину.
7	Оформлення пояснювальної записки, висновків і додатків.	Підготовлено текстову та графічну частину.

РЕФЕРАТ

Пояснювальна записка присвячена розробленню системи EduGraph – веб-застосунку для семантичного пошуку в корпусі освітніх документів на основі LLM-орієнтованих transformer-моделей з інтерактивною графовою навігацією результатів. Робота є розширенням звіту з переддипломної практики: матеріал практики використано як фактичну основу, а зміст доповнено повною аналітичною, функціональною та проєктною частинами.

У роботі розглянуто проблему пошуку в освітніх документах, обґрунтовано потребу в семантичному retrieval-підході, описано ролі користувачів, структуру системи, модель даних, серверну частину на ASP.NET Core Web API, клієнтську частину на React/Vite та FastAPI-сервіс для індексації й пошуку за embeddings. Джерелом корпусу є Google Drive, реляційні метадані зберігаються у SQLite, а векторні подання фрагментів – у LanceDB.

Ключові слова: семантичний пошук, LLM, embeddings, ASP.NET Core, React, FastAPI, Google Drive, LanceDB, графова навігація.

ABSTRACT

The explanatory note describes EduGraph, a web system for semantic search in a corpus of educational documents using LLM-oriented transformer models and interactive graph navigation of results. The work extends the pre-diploma practice report into a full explanatory note with analytical, functional and design sections.

The system uses Google Drive as the document source, ASP.NET Core Web API for authorization and business workflows, React/Vite for the user interface, SQLite for structured metadata and a FastAPI service with SentenceTransformers, LanceDB and CrossEncoder reranking for semantic retrieval.

Keywords: semantic search, LLM, embeddings, ASP.NET Core, React, FastAPI, Google Drive, LanceDB, graph navigation.

ВІДОМІСТЬ РОБОТИ

Пояснювальна записка містить опис кваліфікаційної роботи, виконаної на основі проєкту EduGraph. Документ розвиває матеріали звіту з переддипломної практики: попередній текст було використано як основу для вступу, аналізу предметної області, огляду технологій і опису реалізованих компонентів. У цій версії матеріал розширено до повноцінної структури пояснювальної записки з деталізацією функціональних сценаріїв, архітектури, бази даних, серверної й клієнтської частин, тестування, інструкції користувача та економічно-організаційної частини.

Окремий розділ з охорони праці та безпеки в надзвичайних ситуаціях у цей документ не включено, оскільки він оформлюється як самостійний документ.

Елемент	Стан у записці
Титульний аркуш	Підготовлений як заповнюваний академічний шаблон.
Завдання	Надано змістовну частину для перенесення у форму кафедри.
Реферат	Виконано українською та англійською мовами.
Основні частини	Розширено аналітичну, функціональну та проєктну частини.
Додаток А	Додано альбом графічного матеріалу у вигляді схем.

ЗМІСТ

ВСТУП

1 АНАЛІТИЧНА ЧАСТИНА

- 1.1 Аналіз існуючих типів застосунків
- 1.2 Аналіз типової архітектури
- 1.3 Огляд існуючих комплексних рішень
- 1.4 Аналіз середовищ створення
- 1.5 Огляд ОС і мов програмування
- 1.6 Постановка задачі
- 1.7 Попередній вибір проєктних рішень

2 ФУНКЦІОНАЛЬНА ЧАСТИНА

- 2.1 Проєктування варіантів використання системи
- 2.2 Специфікації основних сценаріїв
- 2.3 Діаграми сценаріїв та взаємодії

3 ПРОЄКТНА ЧАСТИНА

- 3.1 Вибір базових технологій
- 3.2 Визначення функціональних залежностей
- 3.3 Проєктування бази даних
- 3.4 Проєктування серверної частини
- 3.5 Проєктування клієнтської частини
- 3.6 Опис основних класів, методів і змінних
- 3.7 Структура інтерфейсу користувача
- 3.8 Експериментальне тестування
- 3.9 Інструкція користувача
- 3.10 Рекомендації щодо використання

5 ЕКОНОМІЧНО-ОРГАНІЗАЦІЙНА ЧАСТИНА

ВИСНОВКИ

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

ДОДАТКИ

ВСТУП

Переддипломна практика стала фактичною основою для кваліфікаційної роботи, оскільки під час її проходження було систематизовано поточний стан проєкту EduGraph, перевірено відповідність реалізації обраний темі та визначено, які частини потрібно розширити в пояснювальній записці. Якщо звіт з практики описував переважно виконані дії та загальний огляд реалізації, то пояснювальна записка має показати повний шлях від аналізу предметної області до обґрунтування прийнятих проєктних рішень.

Цифрове освітнє середовище університету створює значний обсяг документів, які зберігаються у різних форматах і часто мають складну структуру. До таких документів належать освітні програми, методичні рекомендації, робочі програми дисциплін, інструкції, звіти, таблиці, розклади та допоміжні матеріали. Формально всі ці документи можуть бути доступні через файлове сховище, але доступність файла не означає зручність пошуку потрібного змісту.

Основна проблема полягає в тому, що користувач часто шукає не конкретний файл, а відповідь на змістове питання. Назва документа може не містити потрібного терміна, а релевантний фрагмент може знаходитися всередині великого PDF або DOCX. Якщо пошук виконується тільки за точним збігом слів, система не враховує синоніми, перефразування, контекст і близькість понять. Для освітніх документів це призводить до ситуації, коли матеріал існує, але користувач не знаходить його за власним формулюванням запиту.

Тема кваліфікаційної роботи – «AI-система семантичного пошуку в корпусі освітніх документів на основі LLM з інтерактивною графовою навігацією результатів». Формулювання теми зберігається без зміни, однак у межах пояснювальної записки потрібно коректно пояснити технічну суть реалізації. У поточному стані EduGraph не використовує генеративну чат-модель для створення відповідей. Натомість система реалізує retrieval-частину LLM-орієнтованого підходу: текстові фрагменти перетворюються на

embeddings, запит також перетворюється на embedding, після чого виконується пошук близьких фрагментів і переранжування результатів.

Такий підхід є доцільним для навчального середовища, оскільки він зберігає зв'язок з першоджерелом. Користувач бачить не синтетично згенеровану відповідь, а назву документа, папку, релевантний фрагмент і посилання на оригінальний файл у Google Drive. Це знижує ризик некоректної інтерпретації матеріалу та дозволяє перевіряти знайдену інформацію в контексті повного документа.

Метою роботи є розроблення та обґрунтування веб-системи EduGraph, яка забезпечує синхронізацію корпусу освітніх документів із Google Drive, збереження метаданих у базі даних, побудову семантичного індексу текстових фрагментів, пошук за змістом і подання результатів у вигляді інтерактивного графу. Мета охоплює не тільки програмну реалізацію, а й пояснення того, чому обрана архітектура відповідає задачам предметної області.

Об'єктом дослідження є процес пошуку та навігації в корпусі освітніх документів. Предметом дослідження є програмні засоби синхронізації документів, обробки тексту, побудови векторних представлень, ранжування результатів і графового відображення зв'язків між папками та документами. У практичній реалізації ці засоби представлені трьома компонентами: ASP.NET Core Web API, React-клієнтом і FastAPI-сервісом семантичного пошуку.

Для досягнення мети необхідно виконати комплекс задач. Потрібно проаналізувати предметну область, визначити типові класи застосунків для пошуку документів, обґрунтувати клієнт-серверну архітектуру, спроектувати ролі користувачів, описати сценарії синхронізації та пошуку, визначити структуру бази даних, описати реалізацію серверної та клієнтської частин, а також сформулювати інструкцію користувача й рекомендації щодо подальшого розвитку системи.

Практична значущість роботи полягає у створенні прототипу системи, який може бути використаний як основа для університетського сервісу пошуку по навчальних матеріалах. EduGraph не вимагає повного перенесення

документів у нову систему, а використовує Google Drive як джерело документного корпусу. Це дозволяє доповнити вже наявну інфраструктуру семантичним пошуком і графовою навігацією без руйнування звичного способу зберігання матеріалів.

Пояснювальна записка побудована як розвиток матеріалів звіту з переддипломної практики. У дипломній роботі ці матеріали деталізовано з погляду вимог, варіантів використання, структури даних, графового подання результатів, інтерфейсу та функціональної перевірки системи.

1 АНАЛІТИЧНА ЧАСТИНА

1.1 Аналіз існуючих типів застосунків для пошуку в освітніх документах

Освітні документи можуть зберігатися в різних інформаційних системах, і кожен клас систем має власну логіку пошуку. Найпростішим класом є файлові сховища, до яких належать Google Drive, OneDrive або локальні мережеві каталоги. Вони добре виконують функцію збереження файлів, надання спільного доступу, організації папок і контролю посилань. Для університетського середовища це важливо, оскільки матеріали часто оновлюються різними відповідальними особами й повинні залишатися доступними за стабільними посиланнями.

Недоліком файлових сховищ є те, що їхній пошук орієнтований переважно на назви, метадані або простий повнотекстовий індекс. Якщо документ має загальну назву або містить велику кількість розділів, користувачеві складно зрозуміти, де саме знаходиться потрібний фрагмент. Також файлове сховище не завжди пояснює релевантність результату: користувач бачить файл, але не бачить, який саме уривок відповідає запиту.

Другим класом є системи управління навчанням. Вони добре структурують матеріали за курсами, темами, завданнями й календарем освітнього процесу. Однак LMS зазвичай прив'язана до конкретної дисципліни або групи, а не до наскрізного корпусу документів. Якщо потрібно знайти матеріал, що стосується кількох дисциплін, навчальних планів або нормативних документів, користувач може не знати, в якому курсі або розділі його шукати.

Третій клас становлять системи електронного документообігу. Вони орієнтовані на маршрутизацію документів, погодження, контроль версій і формальні процеси. Такі системи важливі для адміністративної діяльності, але не завжди зручні для студентського або викладацького пошуку навчального змісту. Їхній інтерфейс часто побудований навколо реквізитів документа, а не навколо змістового запиту користувача.

Четвертим класом є повнотекстові пошукові системи. Вони будують індекс термінів і дозволяють швидко знаходити документи, де зустрічаються певні слова. Такі системи можуть бути дуже ефективними для точних запитів, наприклад для пошуку назви нормативного документа або конкретного терміна. Однак вони слабше працюють там, де користувач формулює запит іншими словами, ніж автор документа.

Для освітніх матеріалів ця проблема є типовою. Один і той самий зміст може бути описаний через різні поняття: «результати навчання», «компетентності», «очікувані результати», «програмні результати». Якщо система не враховує семантичну близькість, вона може не знайти релевантний фрагмент або поставити його нижче менш корисного документа. Тому для задачі EduGraph важливо шукати не тільки збіг символів, а й змістову відповідність.

П'ятим класом є системи семантичного пошуку. Вони представляють текст у вигляді числових векторів, які відображають зміст. Запит користувача також перетворюється на вектор, після чого система шукає найближчі фрагменти у векторному просторі. На відміну від класичного пошуку, такий підхід може знайти документ навіть тоді, коли слова запиту й документа не збігаються повністю.

Семантичний пошук особливо корисний тоді, коли корпус складається з довгих документів. Якщо індексувати документ цілком, релевантність може розмиватися через велику кількість тем усередині одного файлу. Тому EduGraph використовує підхід з поділом документа на фрагменти. Кожен chunk індексується окремо, а користувач отримує не тільки назву документа, а й текстовий уривок, який став причиною релевантності.

Окремою вимогою є навігація по результатах. Звичайний список документів показує порядок релевантності, але не показує структуру корпусу. Якщо результати належать до різних папок, користувачу корисно бачити, чи зосереджені вони в одному тематичному розділі або розкидані по різних частинах сховища. Тому EduGraph доповнює пошук графовим поданням, де

папки та документи є вузлами, а зв'язки відображають належність документа до певної частини корпусу.

Таким чином, аналіз існуючих типів застосунків показує, що жоден окремий клас традиційних систем не закриває повністю потребу EduGraph. Файлове сховище потрібне як джерело документів, серверна система потрібна для контролю доступу, semantic search потрібний для змістового пошуку, а графове подання потрібне для навігації. Саме поєднання цих підходів визначає архітектуру розроблюваної системи.

1.2 Аналіз типової архітектури веб-системи семантичного пошуку

Типова веб-система семантичного пошуку складається з кількох логічних шарів. Перший шар – клієнтський інтерфейс, через який користувач вводить запит, переглядає результати та виконує дії відповідно до своєї ролі. Другий шар – backend API, який приймає запити, перевіряє авторизацію, виконує бізнес-логіку та координує роботу інфраструктурних компонентів. Третій шар – сховище структурованих даних, де зберігаються користувачі, документи, стани індексації та службова інформація.

Для системи семантичного пошуку потрібен також окремий індексаційний шар. Він відповідає за перетворення документів у придатний для пошуку вигляд: отримання тексту, нормалізацію, поділ на фрагменти, побудову embeddings і збереження векторів. Цей шар відрізняється від звичайної бізнес-логіки, оскільки використовує ML/NLP-бібліотеки, великі моделі та спеціалізоване векторне сховище.

У EduGraph клієнтським шаром є React SPA. Він не містить логіки синхронізації документів або виконання embeddings, а працює з backend через HTTP. Такий підхід зменшує складність клієнта й не відкриває внутрішню інфраструктуру напряду. Клієнт отримує тільки ті дані, які потрібні для інтерфейсу: список папок, результати пошуку, фрагменти, посилання та інформацію про роль користувача.

Backend-шар реалізований на ASP.NET Core Web API. Він виконує роль центрального координатора. Саме backend знає про користувачів, ролі, заявки,

Google Drive, SQLite і FastAPI-сервіс. Клієнт ніколи не звертається напряму до Python-сервісу, тому всі пошукові операції проходять через контроль авторизації. Це важливо для університетського середовища, де доступ до матеріалів має бути керованим.

Сховище структурованих даних реалізовано через SQLite. У ньому зберігаються користувачі, ролі, заявки на реєстрацію, папки Google Drive, документи, hash вмісту та статуси індексації. SQLite не використовується для векторного пошуку, але є джерелом істини для доменних метаданих. Це розділення дозволяє не змішувати реляційні дані з ML-індексом.

Векторний пошук реалізований окремим FastAPI-сервісом. Він використовує Python-бібліотеки, які є типовими для задач NLP: SentenceTransformers, CrossEncoder, text splitters і LanceDB. Винесення цього сервісу в окремий процес дозволяє змінювати модель embeddings або параметри chunking без зміни ASP.NET Core API та React-клієнта. Також це спрощує майбутнє масштабування ML-частини.

Джерелом документів є Google Drive. Це зовнішній сервіс, який зберігає оригінали документів, папки та посилання. Backend отримує документи через Google Drive API, експортує Google Workspace-файли в оброблюваний формат, читає PDF, DOCX, TXT і CSV, а потім зберігає текст і метадані в локальній базі. Оригінальне посилання зберігається, щоб користувач міг перейти до першоджерела.

Типова взаємодія під час пошуку відбувається послідовно. Користувач вводить запит у React. React викликає endpoint ASP.NET Core API. Backend перевіряє JWT і роль користувача, формує запит до FastAPI-сервісу, отримує ranked результати, перетворює їх у відповідний response DTO та повертає клієнту. React будує граф і відкриває модальне вікно фрагмента за дією користувача.

Типова взаємодія під час синхронізації має іншу послідовність. Backend звертається до Google Drive, обходить папки, отримує файли, парсить текст, обчислює hash, оновлює записи SQLite, визначає змінені або видалені

документи, а потім викликає FastAPI endpoint для upsert або delete векторного індексу. Таким чином, API є координатором між джерелом документів і semantic index.

Архітектура з трьома основними частинами має компроміси. Вона складніша, ніж монолітний застосунок, оскільки потребує налаштування HTTP-взаємодії між сервісами. Водночас вона краще відповідає задачі: frontend, backend і vector-search мають різні технологічні вимоги, різні залежності та різні темпи розвитку. Для дипломного проєкту такий поділ є обґрунтованим, оскільки демонструє сучасний підхід до побудови AI-орієнтованих веб-систем.

Узагальнена архітектура системи EduGraph

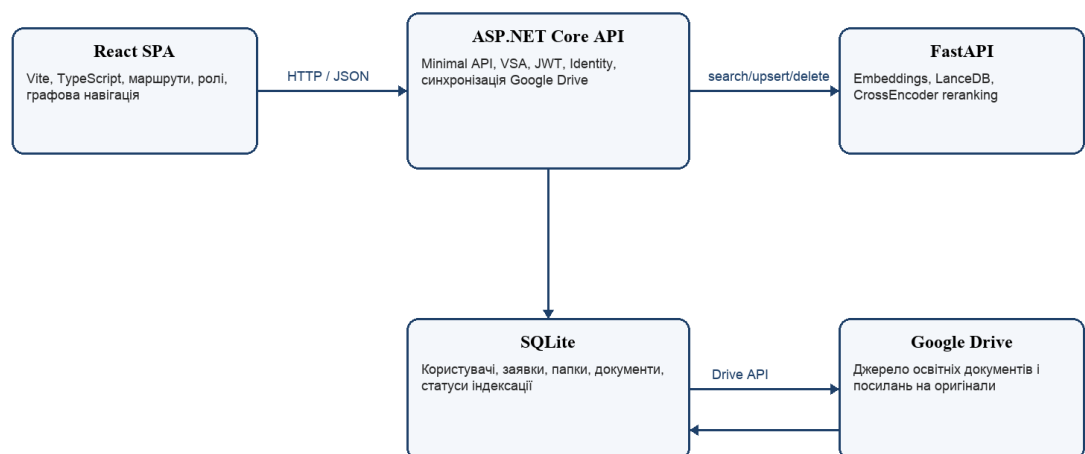


Рисунок 1.1 – Узагальнена архітектура системи EduGraph

1.3 Огляд існуючих комплексних рішень

Комплексні рішення для роботи з документами можна оцінювати за кількома критеріями: підтримка джерел документів, можливість пошуку за змістом, контроль доступу, пояснюваність результатів, складність впровадження, вартість і можливість адаптації під власний інтерфейс. Для EduGraph важливо, щоб рішення не вимагало повної заміни Google Drive, оскільки освітні матеріали вже можуть зберігатися у цьому середовищі.

Google Drive надає базовий пошук і доступ до документів, але не вирішує задачі графової навігації та спеціалізованого semantic retrieval. Його сильна сторона – стабільне зберігання файлів, спільний доступ, структура папок і посилання. Його слабка сторона в контексті EduGraph – обмежена можливість налаштувати алгоритм релевантності, додати власний reranking або показувати користувачеві знайдений chunk.

Elasticsearch або OpenSearch могли б бути альтернативою для повнотекстового пошуку. Такі системи мають розвинений індекс, фільтрацію, сортування й масштабування. Проте для семантичного пошуку потрібна або додаткова vector search конфігурація, або окрема ML-інтеграція. Для бакалаврського прототипу це може бути надмірно складним порівняно з LanceDB, який одразу орієнтований на векторні операції.

Векторні бази даних на кшталт Pinecone, Weaviate, Qdrant або Milvus є потужними рішеннями для production-сценаріїв. Вони можуть масштабуватися, підтримувати фільтри й мати розвинені засоби адміністрування. Однак частина таких рішень потребує окремого сервера або хмарної інфраструктури. Для поточного прототипу EduGraph обрано LanceDB як простіше локальне сховище, достатнє для демонстрації підходу.

LMS-рішення, такі як Moodle або Google Classroom, зручні для організації навчального процесу, але вони не є спеціалізованими системами semantic search по розподіленому корпусу документів. Їхній пошук зазвичай прив'язаний до курсу або конкретного середовища. EduGraph, навпаки, розглядає документи як єдиний корпус і дозволяє шукати по ньому незалежно від того, в якій папці або дисципліні вони розташовані.

Системи RAG можуть бути ближчими до теми LLM. Вони поєднують retrieval і генерацію відповіді. Проте для освітнього середовища першим критичним кроком є надійне знаходження джерельних фрагментів. Якщо retrieval працює неточно, генеративний шар може лише посилити помилки. Тому в EduGraph реалізовано саме retrieval-основу: система знаходить

документ і фрагмент, а генеративне пояснення може бути додано в майбутньому.

Порівняння комплексних рішень показує, що EduGraph займає проміжну позицію між файловим сховищем, search engine і RAG-системою. Він не дублює Google Drive, не замінює LMS і не намагається одразу створювати генеративні відповіді. Його призначення – додати до наявного корпусу семантичний шар, контрольований backend і графову навігацію.

Такий вибір є практичним для дипломної роботи. Він дозволяє показати повний цикл: отримання документів із зовнішнього джерела, парсинг, збереження метаданих, побудову vector index, пошук, reranking, авторизацію та frontend-візуалізацію. Кожен компонент має чітку роль, а вся система залишається достатньо компактною для розроблення й пояснення в межах кваліфікаційної роботи.

Клас рішення	Сильні сторони	Обмеження для EduGraph
Файлове сховище	Зберігання, папки, посилання, спільний доступ.	Недостатня семантична релевантність і відсутність спеціального графу.
LMS	Організація курсів і навчального процесу.	Обмежений наскрізний пошук по всьому корпусу документів.
Повнотекстовий пошук	Швидкий пошук за точними словами.	Гірше працює з переформульованими запитами.
Vector search	Пошук за змістовою близькістю.	Потребує індексації, embeddings і налаштування якості.

1.4 Аналіз середовищ створення застосунку

Розроблення EduGraph потребує використання кількох середовищ, оскільки система складається з .NET backend, TypeScript frontend і Python

vector-search service. Для ASP.NET Core частини доцільно використовувати JetBrains Rider або Visual Studio, оскільки вони забезпечують повну підтримку C#, проєктів .NET, NuGet-пакетів, Entity Framework Core і запуску Web API. Важливими можливостями є навігація по solution, перевірка типів, інтеграція з Git і запуск профілів розроблення.

Для клієнтської частини з React і Vite достатньо Visual Studio Code або WebStorm. Таке середовище має підтримувати TypeScript, JSX/TSX, ESLint, npm scripts і швидкий запуск dev server. Оскільки frontend EduGraph використовує рольову маршрутизацію, API-модулі й графову візуалізацію, важливо мати зручну навігацію між сторінками, компонентами, типами DTO та утилітами.

Python-сервіс семантичного пошуку зручніше розробляти в PyCharm або Visual Studio Code з налаштованим віртуальним середовищем. Для такого сервісу критичні залежності sentence-transformers, FastAPI, LanceDB і text splitters. Середовище має дозволяти запускати сервіс окремо, перевіряти endpoint-и, бачити логи ініціалізації моделей і контролювати версії пакетів.

У монорепозіторії важливо мати єдиний підхід до контролю версій. Git дозволяє зберігати історію змін frontend, backend і vector-search service разом. Це спрощує відстеження архітектурних змін, оскільки зміна API-контракту може одночасно вимагати змін у C# endpoint, TypeScript API-модулі та Python DTO.

Для документування API використовується OpenAPI/Scalar. Це важливо не тільки для розроблення, а й для пояснювальної записки, оскільки endpoint-и можна описувати як формальні контракти. Наявність OpenAPI-опису дозволяє перевіряти, які маршрути надає backend і які response-типи очікує frontend.

Окремим інструментальним аспектом є робота з базою даних. EF Core migrations дають можливість фіксувати еволюцію фізичної структури SQLite. Для дипломної роботи це корисно, бо можна показати, що модель даних не

існує тільки в тексті, а має реальну міграційну історію, пов'язану з документами, папками, користувачами й індексацією.

Для перевірки frontend застосунку використовується стандартний Vite dev server. Він дозволяє швидко бачити зміни інтерфейсу, перевіряти маршрути, стани графа та взаємодію з API через проху. Оскільки UI має різні стани – завантаження папок, пошук, порожній результат, помилка, перегляд документа – інтерактивна перевірка є обов'язковою частиною розроблення.

Для перевірки Python-сервісу важливо мати окремі HTTP-запити до /health, /documents/upsert, /documents/delete і /search. У проєкті є test_main.http, однак його приклади не повністю відповідають поточному API, тому перед фінальним поданням бажано оновити цей файл або не посилатися на нього як на актуальний тестовий набір.

1.5 Огляд ОС, мов програмування та середовищ виконання

Основні технології EduGraph є кросплатформними. ASP.NET Core може виконуватися на Windows, Linux і macOS, що дозволяє розробляти систему локально та переносити її на сервер. React/Vite виконується через Node.js і браузер, тому клієнтська частина також не прив'язана до конкретної операційної системи. FastAPI працює в Python-середовищі й може бути запущений як локально, так і в контейнері або окремому серверному процесі.

C# обрано для backend через сильну типізацію, зрілу екосистему ASP.NET Core, інтеграцію з Identity, Entity Framework Core і зручну побудову Minimal API. Для системи, де важливі ролі, авторизація, транзакції та фонові процеси, C# дає стабільну платформу з розвиненим інструментарієм.

TypeScript обрано для frontend, оскільки він дозволяє явно описувати API DTO, ролі, стани сторінок і структуру графу. У проєкті це важливо, бо frontend отримує від backend результати пошуку з полями title, url, folderName, content, score, chunkId і chunkIndex. Без типізації такі контракти легше пошкодити під час змін.

Python обрано для vector-search сервісу через доступність бібліотек машинного навчання та обробки природної мови. SentenceTransformers,

CrossEncoder і LanceDB природно інтегруються з Python-кодом. Реалізовувати цю частину всередині C#-проєкту було б складніше й менш гнучко.

SQLite використано як легку СУБД для прототипу. Її перевага полягає у простому розгортанні та відсутності потреби в окремому сервері бази даних. Для дипломної роботи це дозволяє зосередитися на архітектурі й функціональності, а не на адмініструванні БД. У майбутньому за потреби SQLite може бути замінена на PostgreSQL або іншу серверну СУБД.

LanceDB використано як векторне сховище для embeddings. На відміну від реляційної БД, воно орієнтоване на пошук найближчих векторів. Це відповідає задачі semantic retrieval, де основною операцією є не SQL-запит по полях, а пошук за близькістю у векторному просторі.

Google Drive API використовується як інтерфейс до зовнішнього документного корпусу. Такий вибір дозволяє працювати з реальною структурою папок і документів без створення власного файлового сховища. Система читає документи, але не повинна змінювати оригінали, що відповідає принципу безпечної інтеграції з навчальними матеріалами.

Таким чином, поєднання C#, TypeScript і Python не є випадковим. Кожна мова використовується там, де вона найкраще відповідає задачі: C# – для backend і доменної логіки, TypeScript – для типізованого frontend, Python – для NLP і vector search.

1.6 Постановка задачі

Задача роботи полягає у створенні системи, яка дозволяє користувачам з різними ролями виконувати семантичний пошук у корпусі освітніх документів, отримувати релевантні фрагменти й переходити до оригінальних файлів. Система має бути інтегрована з Google Drive, оскільки саме там зберігається первинний корпус документів. Водночас система має мати власну базу метаданих і власний семантичний індекс.

Першою групою вимог є вимоги до користувачів і ролей. Система повинна підтримувати студентів, викладачів, адміністраторів і суперадміністраторів. Студент повинен мати доступ до пошуку. Викладач і

адміністратор повинні мати розширені можливості щодо заявок і користувачів. Суперадміністратор повинен мати доступ до керування адміністраторами.

Другою групою вимог є вимоги до реєстрації. Користувач не повинен створювати обліковий запис напряду без контролю. Замість цього він подає заявку, яка потім підтверджується або відхиляється уповноваженою особою. Такий підхід відповідає навчальному середовищу, де доступ до внутрішніх матеріалів має бути контрольованим.

Третьою групою вимог є вимоги до синхронізації. Система повинна отримувати структуру папок і документів з Google Drive, обробляти документи різних форматів, витягувати текст, зберігати метадані, обчислювати hash вмісту та визначати, які документи потребують повторної індексації. Важливо також коректно реагувати на видалення документів у Google Drive.

Четвертою групою вимог є вимоги до семантичного пошуку. Система повинна ділити текст на фрагменти, будувати embeddings, зберігати їх у vector store, приймати запит користувача, знаходити кандидатів за cosine similarity, застосовувати reranking і повертати обмежену кількість найбільш релевантних документів. Результат повинен містити не тільки назву документа, а й фрагмент тексту.

П'ятою групою вимог є вимоги до інтерфейсу. Користувач повинен бачити не лише список результатів, а й графову структуру, яка показує зв'язки між кореневою папкою, тематичними папками й документами. Інтерфейс має підтримувати стани завантаження, помилки, порожнього результату та перегляду документа.

Нефункціональні вимоги включають модульність, розширюваність, контроль доступу, зрозумілість інтерфейсу, можливість локального запуску та відносну незалежність ML-компонента від основного backend. Система має бути побудована так, щоб у майбутньому можна було змінити модель

embeddings, додати фільтри, підключити генеративний LLM-шар або замінити СУБД.

Обмеженням поточної роботи є те, що кількісна оцінка якості пошуку за метриками precision, recall, MRR або nDCG не виконується в межах цієї версії. Для такої оцінки потрібен окремий набір контрольних запитів і еталонних відповідей. У записці описується функціональне тестування та обґрунтування алгоритму, але не повний information retrieval benchmark.

1.7 Попередній вибір проєктних рішень

На основі аналізу предметної області було обрано архітектуру з трьома основними компонентами: frontend, backend API та vector-search сервіс. Такий поділ відповідає різній природі задач. Frontend відповідає за взаємодію з користувачем. Backend відповідає за авторизацію, доменні правила, Google Drive і базу даних. Vector-search сервіс відповідає за ML/NLP-частину.

Для backend обрано ASP.NET Core Minimal API. Це дозволяє будувати легкий HTTP API без надлишкової MVC-структури. У проєкті використано Vertical Slice Architecture, де кожна функція оформлюється як окремий файл з власними request, validator, endpoint і handler. Це полегшує підтримку, оскільки логіка конкретного сценарію знаходиться поруч.

Для контролю користувачів обрано ASP.NET Core Identity і JWT Bearer. Identity забезпечує роботу з користувачами, ролями та перевіркою пароля, а JWT дозволяє frontend додавати токен до кожного запиту. Такий підхід добре підходить для SPA, де frontend і backend взаємодіють через HTTP.

Для збереження даних обрано EF Core і SQLite. EF Core дає можливість працювати з сутностями, міграціями та LINQ-запитами. SQLite є достатньою для прототипу, оскільки система не вимагає складної серверної БД на етапі дипломної реалізації. Водночас використання EF Core залишає можливість майбутньої міграції на іншу СУБД.

Для Google Drive обрано офіційний API та service account з read-only доступом. Це обмежує ризик випадкової зміни джерельних документів.

Система не редагує Google Drive, а тільки читає папки, документи, MIME-типи, посилання та вміст файлів.

Для семантичного пошуку обрано FastAPI, SentenceTransformers, LanceDB і CrossEncoder. FastAPI забезпечує простий HTTP-інтерфейс. SentenceTransformers створює embeddings для запитів і фрагментів. LanceDB виконує vector search. CrossEncoder додає другий етап оцінювання, який точніше порівнює запит і кандидатний фрагмент.

Для frontend обрано React, TypeScript і Vite. React дозволяє будувати компонентну структуру, TypeScript зменшує ризик помилок у DTO, а Vite забезпечує швидку розробку. Для графу використано react-force-graph-2d, але значна частина логіки позиціонування та вигляду вузлів реалізована на рівні сторінки пошуку.

Попередній вибір проєктних рішень показує, що система не є набором випадкових технологій. Кожен компонент закриває власну зону відповідальності, а взаємодія між ними будується через зрозумілі HTTP-контракти. Це робить EduGraph придатним для пояснення, демонстрації та подальшого розвитку.

2 ФУНКЦІОНАЛЬНА ЧАСТИНА

2.1 Проектування варіантів використання системи

Функціональна модель EduGraph описує не окремі екрани, а повні сценарії роботи користувачів і зовнішніх сервісів. Межами системи є React-клієнт, ASP.NET Core API, SQLite-сховище, інтеграція з Google Drive та FastAPI-сервіс семантичного пошуку. Google Drive API і сервіс семантичного пошуку розглядаються як зовнішні учасники сценаріїв, оскільки вони виконують спеціалізовані дії поза інтерфейсом користувача.

Таблиця 2.1 – Актори системи EduGraph

Актор	Роль у системі	Основні операції	Обмеження
Неавторизований користувач	Кандидат на доступ	Вхід до системи, подання заявки	Не має доступу до пошуку та документів
Студент	Основний користувач пошуку	Пошук, перегляд графа, відкриття фрагмента й оригіналу	Не керує заявками, користувачами та синхронізацією
Викладач	Користувач з розширеним доступом	Пошук, заявки, створення студентів, запуск синхронізації	Не керує адміністраторами
Адміністратор	Керування користувачами й заявками	Пошук, заявки, студенти, викладачі, синхронізація	Не створює інших адміністраторів
Суперадміністратор	Повний адміністративний доступ	Усі адміністративні сценарії, зокрема адміністратори	Використовується обмеженим колом осіб
Google Drive API	Зовнішнє джерело корпусу	Повернення папок, файлів, посилань і вмісту	Не відповідає за авторизацію користувачів EduGraph

Сервіс семантичного пошуку	Зовнішній ML/NLP-компонент	Індексація фрагментів, vector search, reranking	Не керує користувачами й ролями
----------------------------	----------------------------	---	---------------------------------

Таблиця 2.2 – Матриця доступу до варіантів використання

Варіант використання	Student	Teacher	Admin	Super Admin	Сервіс
Увійти до системи	+	+	+	+	ASP.NET Identity
Подати заявку на реєстрацію	до створення ролі	до створення ролі	-	-	Auth API
Переглянути структуру папок	+	+	+	+	Google Drive API
Виконати семантичний пошук	+	+	+	+	FastAPI, LanceDB
Переглянути фрагмент документа	+	+	+	+	React modal
Відкрити оригінал у Google Drive	+	+	+	+	Google Drive URL
Опрацювати заявки	-	+	+	+	SignUpApplications
Керувати студентами	-	+	+	+	Students API

Керувати викладачами	-	-	+	+	Teachers API
Керувати адміністраторами	-	-	-	+	Admins API
Запустити синхронізацію Google Drive	-	+	+	+	Sync queue
Індексувати документи	-	-	-	-	Фонові служба, FastAPI

Загальна діаграма варіантів використання відображає, що центральним сценарієм є пошук документів, але він залежить від попередніх сценаріїв авторизації, синхронізації корпусу та індексації текстових фрагментів.

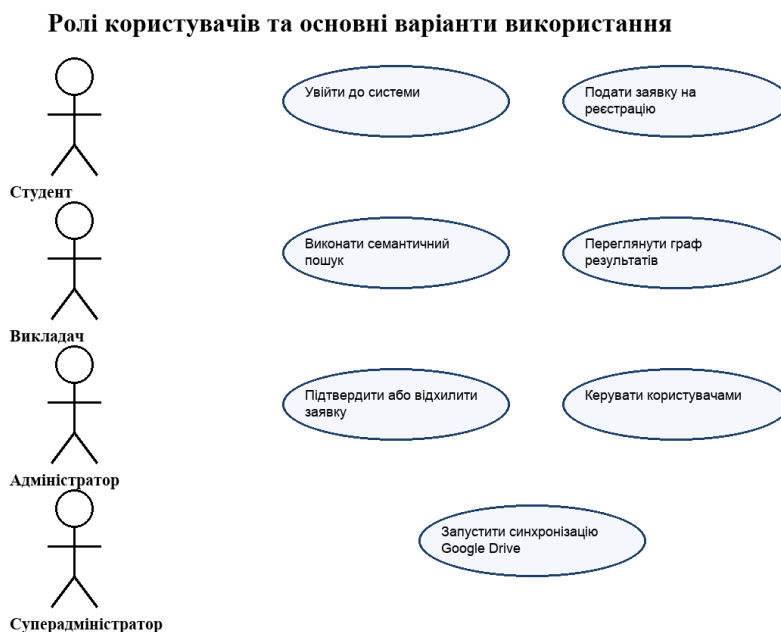


Рисунок 2.1 – Загальна діаграма варіантів використання EduGraph

2.2 Специфікації основних сценаріїв

Для основних варіантів використання сформовано текстові специфікації. Вони потрібні для перевірки повноти функціональних вимог: кожен сценарій має актора, передумови, основний потік, альтернативи, помилки та результат.

Таблиця 2.3 – Специфікація UC-01 Увійти до системи

Поле	Зміст
Основний актор	Авторизований користувач однієї з ролей
Передумови	Користувач має створений обліковий запис; логін і пароль введені у форму входу.
Основний сценарій	Система перевіряє дані через Identity, оновлює дату останнього входу, формує JWT з role claim і перенаправляє користувача на маршрут його ролі.
Альтернативи та помилки	Неправильний пароль, порожні поля, заблокований або відсутній користувач, недійсний токен під час наступних запитів.
Постумова	Користувач отримує доступ тільки до дозволених сторінок.

Таблиця 2.4 – Специфікація UC-02 Подати заявку на реєстрацію

Поле	Зміст
Основний актор	Неавторизований користувач
Передумови	Користувач відкрив сторінку реєстрації та обрав тип майбутньої ролі.
Основний сценарій	Користувач вводить логін, ПІБ, пароль, підтвердження пароля та, для студента, групу. Backend валідує дані й створює заявку зі статусом Pending.

Альтернативи та помилки	Логін уже зайнятий, існує активна заявка з таким логіном, пароль не підтверджено, ПІБ не відповідає формату.
Постумова	Заявка зберігається для подальшого рішення уповноваженої ролі.

Таблиця 2.5 – Специфікація UC-03 Опрацювати заявку на реєстрацію

Поле	Зміст
Основний актор	Викладач, адміністратор або суперадміністратор
Передумови	У системі є заявка зі статусом Pending.
Основний сценарій	Користувач відкриває список заявок, переглядає дані й натискає «Схвалити» або «Відхилити». При схваленні створюється User і призначається роль; при відхиленні змінюється статус заявки.
Альтернативи та помилки	Заявку не знайдено, заявка вже оброблена, створення користувача завершилося помилкою Identity.
Постумова	Заявка переходить у фінальний стан Approved або Rejected.

Таблиця 2.6 – Специфікація UC-04 Виконати семантичний пошук документів

Поле	Зміст
Основний актор	Будь-який авторизований користувач
Передумови	Користувач авторизований, структура папок завантажена, індекс документів існує або готовий до пошуку.

Основний сценарій	Користувач вводить запит, frontend викликає endpoint пошуку, backend валідує Query, FastAPI створює query embedding, виконує пошук у LanceDB, застосовує CrossEncoder reranking і повертає результати.
Альтернативи та помилки	Порожній запит, відсутність результатів, недоступність FastAPI, помилка vector store або некоректне посилання Google Drive.
Постумова	На екрані формується граф із папками та знайденими документами.

Таблиця 2.7 – Специфікація UC-05 Переглянути знайдений фрагмент документа

Поле	Зміст
Основний актор	Користувач, який отримав результати пошуку
Передумови	У графі є вузол документа з title, content, url і folderName.
Основний сценарій	Користувач натискає на вузол документа. Frontend відкриває модальне вікно, показує релевантний фрагмент і кнопку переходу до оригіналу в Google Drive.
Альтернативи та помилки	Документ не має безпечного URL; користувач закриває modal без переходу; фрагмент занадто короткий або містить шум після парсингу.
Постумова	Дані не змінюються; користувач отримує контекст релевантності результату.

Таблиця 2.8 – Специфікація UC-06 Синхронізувати Google Drive

Поле	Зміст
Основний актор	Уповноважений користувач або фонові служба
Передумови	Налаштований root folder Google Drive і доступ service account.
Основний сценарій	Система ставить задачу синхронізації в чергу, отримує папки й документи, витягує текст, оновлює SQLite, видаляє відсутні документи з індексу та відправляє змінені документи до FastAPI.
Альтернативи та помилки	Google Drive недоступний, root folder не знайдено, файл не має текстового шару, FastAPI повертає помилку індексації.
Постумова	Корпус і векторний індекс наближаються до актуального стану Google Drive.

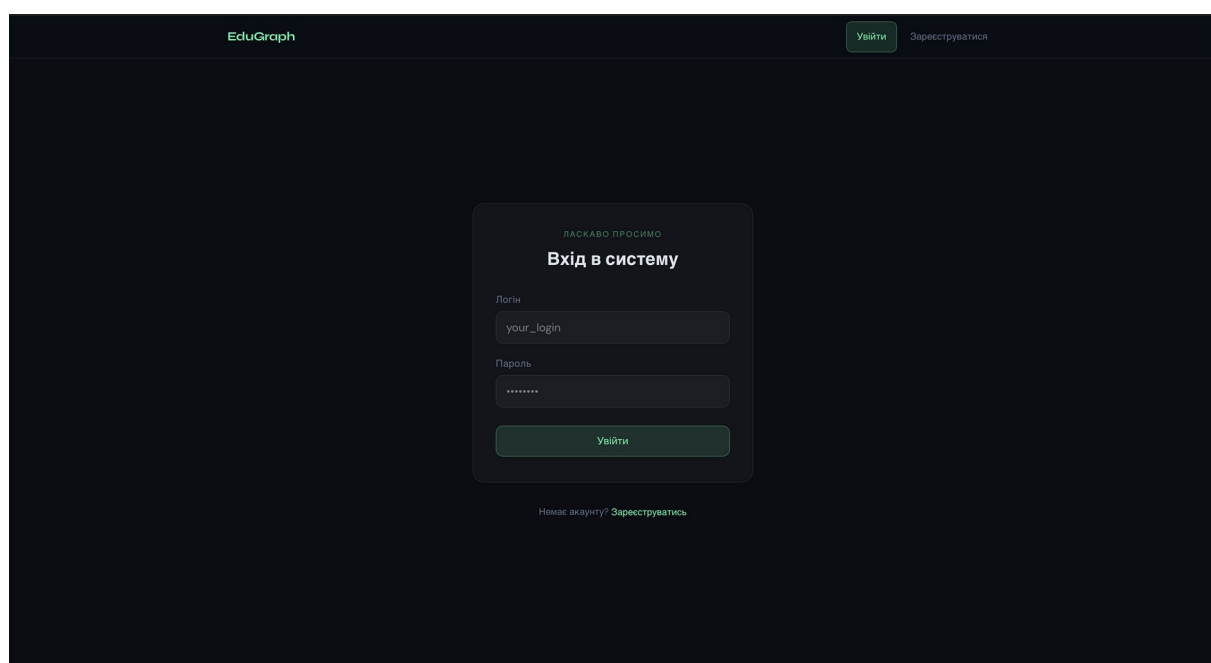


Рисунок 2.2 – Сторінка входу користувача в систему EduGraph

The screenshot shows the 'Заявка на реєстрацію' (Registration Application) form for a new user. The form is titled 'НОВИЙ АКАУНТ' (New Account). It includes fields for 'Логін' (Login) with the placeholder 'your_login', 'ПІБ' (Full Name) with the value 'Іванов Іван Іванович', 'Пароль' (Password) and 'Підтвердьте пароль' (Confirm Password) both masked with dots, and 'Роль' (Role) with radio buttons for 'Студент' (Student) and 'Викладач' (Lecturer). The 'Група' (Group) field contains 'ІО-21'. A 'Надіслати заявку' (Send Application) button is at the bottom.

Рисунок 2.3 – Подання заявки на реєстрацію студентом

The screenshot shows the 'Заявки на реєстрацію' (Registration Applications) section in the admin interface. It features a table with columns: ID, ПІБ, ТИП, ГРУПА, and ДІЇ. The table contains one entry for ID 3, belonging to 'Іванов Іван Іванович', with the role 'Студент' and group 'ІО-22-1'. Action buttons 'Схвалити' (Approve) and 'Відхилити' (Reject) are available for this entry. Navigation controls at the bottom show '← Попередня', '1/1', and 'Наступна →'.

ID	ПІБ	ТИП	ГРУПА	ДІЇ
3	Іванов Іван Іванович	Студент	ІО-22-1	Схвалити Відхилити

Рисунок 2.4 – Перегляд і оброблення заявок на реєстрацію

2.3 Діаграми сценаріїв та взаємодії

Діаграми сценаріїв показують, як варіанти використання переходять у взаємодію компонентів. Для EduGraph важливо окремо показати синхронізацію, пошук і життєвий цикл заявки, тому що ці процеси виконуються різними частинами системи.

Конвеєр синхронізації, індексації та пошуку

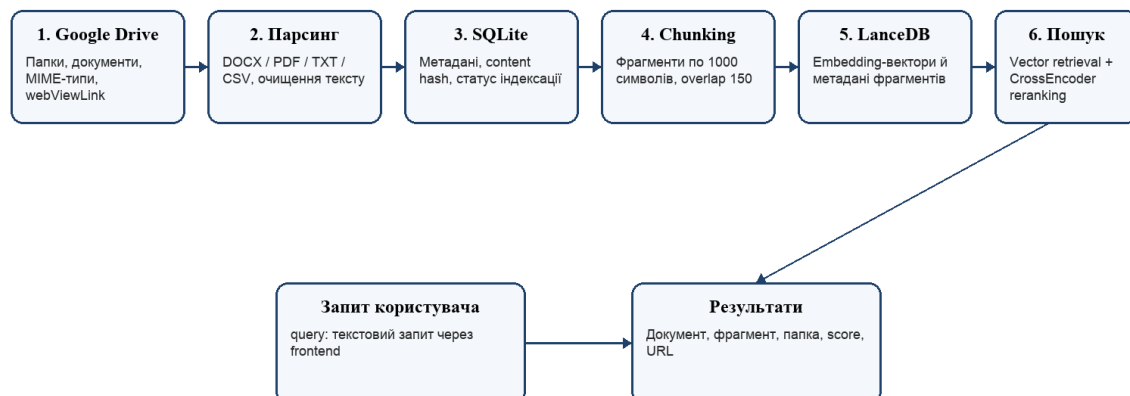


Рисунок 2.5 – Конвеєр синхронізації, індексації та пошуку

Послідовність виконання семантичного пошуку

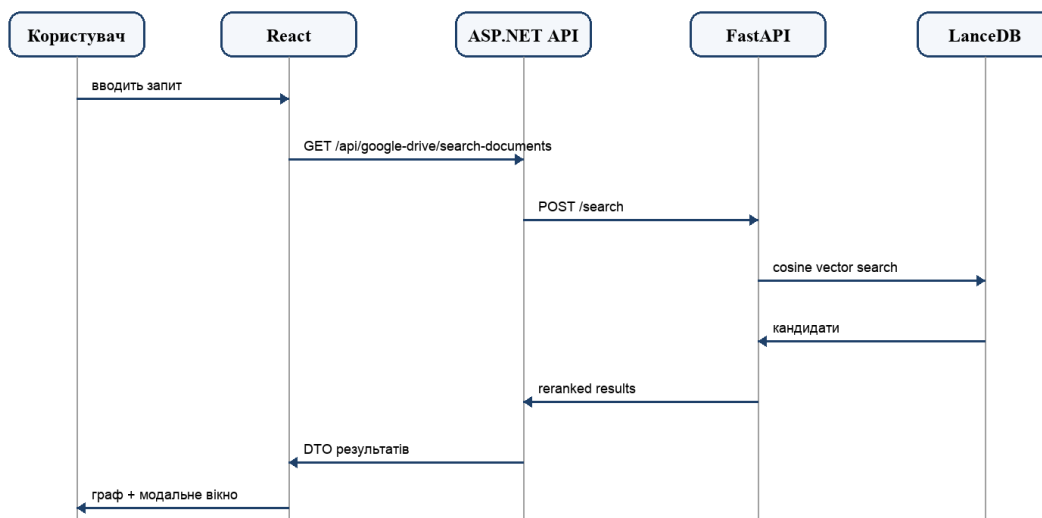


Рисунок 2.6 – Діаграма послідовності семантичного пошуку

Діаграма станів заявки на реєстрацію

Стан заявки змінюється тільки після рішення уповноваженого користувача.



Рисунок 2.7 – Діаграма станів заявки на реєстрацію

Таблиця 2.9 – Зв'язок сценаріїв із реалізацією

Сценарій	Frontend	Backend	Дані / зовнішній сервіс
Вхід	LoginPage, auth utilities	Auth/LogIn.cs	User, JWT
Заявка	SignUpPage	Auth/SignUp.cs	SignUpApplication
Оброблення заявки	SignUpApplicationsPage	Approve/RejectSignUpApplication.cs	Status: Pending, Approved, Rejected
Пошук	StudentSearchPage	SearchDocuments.cs	FastAPI /search, LanceDB
Перегляд фрагмента	DocumentModal	SearchDocuments.Response	title, content, url, folderName
Синхронізація	GoogleDriveControls	SyncGoogleDrive.cs, GoogleDriveSyncService.cs	Google Drive API, UniversityDocument

3 ПРОЄКТНА ЧАСТИНА

3.1 Вибір базових технологій, платформ і стандартів розробки

Вибір технологій у EduGraph визначався не популярністю окремих інструментів, а функціональними потребами системи. Потрібно було поєднати web API, контроль доступу, синхронізацію із зовнішнім сховищем, обробку документів, векторний пошук і інтерактивний інтерфейс. Один технологічний стек не закривав усі ці задачі однаково добре, тому систему спроектовано як комбінацію спеціалізованих компонентів.

ASP.NET Core Web API обрано для серверної частини через підтримку Minimal API, dependency injection, middleware, авторизації, Identity, OpenAPI та фонових служб. Для дипломного проєкту важливо, що ASP.NET Core дозволяє явно описати маршрути й політики доступу, а також має зрілу екосистему для роботи з БД і JWT.

Entity Framework Core використовується як ORM. Він дозволяє працювати з доменними сутностями через C#-код і водночас підтримувати фізичну структуру БД через migrations. Це зручно для проєкту, де модель даних змінювалася під час розвитку: додавалися документи, папки, статуси індексації та індекси.

React і TypeScript обрані для клієнтської частини. React добре підходить для інтерфейсів зі складним станом, а сторінка пошуку має багато станів: початкове завантаження папок, готовність до пошуку, виконання запиту, порожній результат, помилка, показ модального вікна. TypeScript дозволяє формалізувати типи відповідей backend і структуру графу.

FastAPI обрано для Python-сервісу через простоту створення HTTP endpoint-ів і природну інтеграцію з Pydantic. Для ML/NLP-компонента це важливо, оскільки service повинен приймати структуровані запити upsert, delete і search, а також повертати типізовані результати. FastAPI дає достатню продуктивність для прототипу й зручну модель розроблення.

SentenceTransformers використовується для побудови embeddings. У системі застосовується intfloat/multilingual-e5-base, що відповідає

багатомовному характеру освітніх матеріалів і запитів. CrossEncoder використовується як другий етап ранжування, оскільки embedding similarity швидко знаходить кандидатів, але не завжди дає найкращий порядок результатів.

LanceDB обрано як локальне векторне сховище. Для бакалаврського проєкту це практичний компроміс між функціональністю й простотою. Воно дозволяє зберігати vectors разом з метаданими chunk-ів і виконувати cosine search без розгортання окремого великого search cluster.

Google Drive API є джерелом документів. Його використання дозволяє працювати з реальною структурою освітніх матеріалів. Замість того щоб створювати нове сховище, EduGraph додає шар індексації над уже наявним сховищем. Це зменшує організаційний бар'єр для впровадження.

3.2 Визначення функціональних залежностей

Функціональні залежності системи визначають, які компоненти повинні взаємодіяти для виконання конкретного сценарію. Найважливішою залежністю є залежність frontend від backend API. React-клієнт не має прямого доступу ні до SQLite, ні до Google Drive API, ні до FastAPI-сервісу. Він працює тільки з контрактами backend, що дозволяє зберегти контроль доступу й не розкривати внутрішні адреси сервісів.

Backend залежить від SQLite, оскільки саме там зберігається доменний стан системи. Без БД неможливо підтримувати користувачів, ролі, заявки, папки, документи та статуси індексації. Водночас SQLite не виконує vector search, тому backend має другу залежність – FastAPI-сервіс.

Залежність backend від Google Drive API є зовнішньою. Вона потрібна для синхронізації корпусу документів. Якщо Google Drive недоступний, система може зберігати вже проіндексовані документи, але не зможе оновити корпус. Тому синхронізація має бути відокремлена від звичайного пошуку: користувач може шукати в уже наявному індексі навіть тоді, коли оновлення тимчасово неможливе.

FastAPI-сервіс залежить від LanceDB і моделей SentenceTransformers. LanceDB зберігає вектори, а моделі створюють embeddings і reranking scores. Ця залежність є ресурсно чутливою: моделі завантажуються при старті, а обробка великих документів може потребувати часу. Тому backend надсилає документи batch-ами, а не намагається індексувати весь корпус одним великим запитом.

Окремою залежністю є залежність між UniversityDocument і vector index. У SQLite документ має ContentHash і SearchIndexStatus. У LanceDB зберігаються chunk-и з document_id. Якщо документ змінюється, його старі chunk-и мають бути видалені, а нові додані. Цю узгодженість забезпечує GoogleDriveSyncService через upsert.

Функціональні залежності також впливають на відмовостійкість. Якщо vector-search сервіс повертає помилку, backend не повинен втрачати документ. Він позначає індексацію як Failed, що дозволяє повторити процес. Якщо frontend отримує помилку пошуку, він показує повідомлення користувачеві. Якщо документ видалено з Google Drive, backend видаляє його з SQLite і vector index.

Компонент	Залежності	Призначення
React SPA	ASP.NET Core API	Виклик auth, users, Google Drive і search endpoint-ів.
ASP.NET Core API	SQLite	Збереження користувачів, заявок, документів і папок.
ASP.NET Core API	Google Drive API	Отримання структури папок, файлів і посилань.
ASP.NET Core API	FastAPI	Пошук, upsert і delete документів у vector index.
FastAPI	SentenceTransformers, LanceDB	Embeddings, vector search і reranking.

3.3 Проектування бази даних

База даних EduGraph призначена для збереження структурованих даних, які потрібні для контролю доступу, синхронізації та зв'язку між документами й пошуковим індексом. Вона не є основним сховищем файлів і не зберігає embeddings. Такий поділ дозволяє уникнути змішування різних типів даних: реляційні сутності зберігаються в SQLite, а векторні подання – у LanceDB.

3.3.1 Вибір СУБД

Для проєкту обрано SQLite. Основна причина – відповідність масштабу прототипу. SQLite не потребує окремого серверного процесу, легко переноситься разом з проєктом і достатньо добре підходить для збереження метаданих. Для дипломної роботи це практично, оскільки основний акцент робиться на архітектурі semantic search, а не на адмініструванні великої БД.

SQLite також добре інтегрується з Entity Framework Core. Міграції дозволяють фіксувати еволюцію структури, а LINQ-запити спрощують роботу з даними. У проєкті це використовується для пошуку користувачів, вибірки документів зі статусом NotIndexed, отримання папок і видалення записів, яких більше немає в Google Drive.

Обмеження SQLite полягає в тому, що для великого production-навантаження може знадобитися серверна СУБД. Проте поточна задача – прототип і дипломна демонстрація – не вимагає складного кластерного зберігання. Якщо система буде розвиватися, EF Core дозволить перенести модель на PostgreSQL або іншу СУБД з меншими змінами в доменній логіці.

3.3.2 Опис логічної структури БД

Логічна структура БД складається з кількох груп сутностей. Перша група – користувачі та ролі. Користувач має login, тип користувача, групу для студента та дату останнього входу. Ролі реалізовані через Identity, що дозволяє використовувати стандартні механізми ASP.NET Core для перевірки прав.

Друга група – заявки на реєстрацію. SignUpApplication відокремлює намір користувача зареєструватися від фактичного створення облікового

запису. Це важливо для контрольованого доступу. Заявка має статус, який може змінюватися після рішення уповноваженого користувача.

Третя група – папки Google Drive. UniversityFolder зберігає зовнішній GoogleDriveId, назву, посилання та ознаку IsMain для кореневої папки. Ці дані потрібні frontend, щоб будувати граф навіть до виконання пошуку.

Четверта група – документи. UniversityDocument зберігає назву, текстовий вміст, посилання, GoogleDriveId, ContentHash, FolderName, SearchIndexStatus, SearchIndexedAt і SearchIndexError. Саме ця сутність є мостом між Google Drive, SQLite і LanceDB.

П’ята група не є таблицями SQLite, але логічно пов’язана з документами: vector chunks у LanceDB. Кожен chunk має document_id, chunk_id, chunk_index, title, content, url, folder_name, content_hash і vector. Це окремий індекс, але він залежить від GoogleDriveId документа.

Логічна модель даних EduGraph

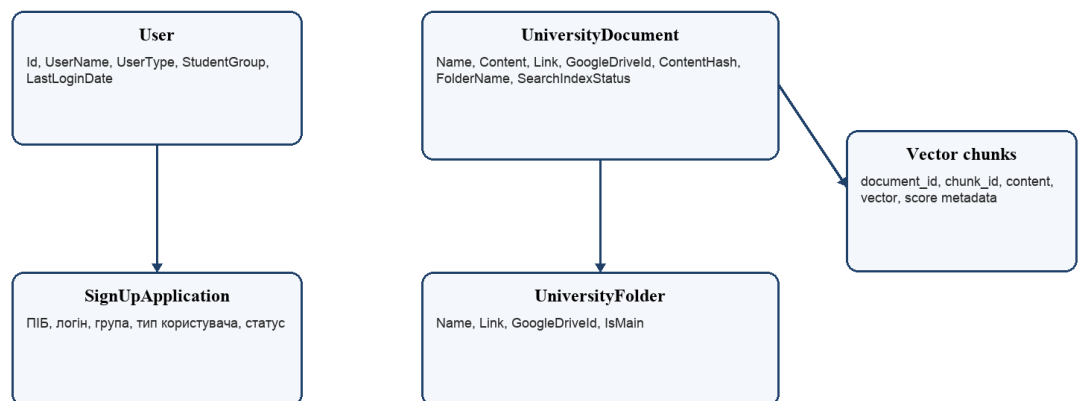


Рисунок 3.1 – Логічна модель даних EduGraph

3.3.3 Опис фізичної структури БД

Фізична структура БД формується EF Core migrations. Міграції в проєкті показують еволюцію системи: початкова структура, додавання сутності документа, зміни документа, додавання folderName, додавання властивостей

індексації, сутності папки та індексів. Це підтверджує, що модель даних розвивалася разом з функціональністю.

Таблиці Identity забезпечують збереження користувачів, ролей і зв'язків між ними. Власна сутність User розширює стандартну модель користувача додатковими полями, потрібними предметній області. Це дозволяє не створювати паралельну систему авторизації, а використовувати перевірений механізм Identity.

Таблиця UniversityDocuments має бути достатньо інформативною для повторної індексації. Вона зберігає не тільки назву й посилання, а й повний текстовий вміст, hash і статус. Завдяки цьому backend може визначити, які документи потрібно відправити у vector-search сервіс, і не індексувати повторно все без потреби.

Фізична структура також повинна підтримувати видалення. Якщо документ зник з Google Drive, його GoogleDriveId більше не присутній у синхронізованому наборі. Backend знаходить такі записи, видаляє їх з vector index і після цього видаляє з SQLite. Така послідовність зменшує ризик залишити в пошуку неактуальні документи.

Сутність	Основні поля	Призначення
User	Id, UserName, UserType, StudentGroup, LastLoginDate	Облікові записи та рольова модель.
SignUpApplication	Name, Login, UserType, StudentGroup, Status	Заявки на створення облікового запису.
UniversityFolder	Name, Link, GoogleDriveId, IsMain	Папки Google Drive та коренева папка.
UniversityDocument	Name, Content, Link, GoogleDriveId, ContentHash, SearchIndexStatus	Документи, текст і стан індексації.
Vector chunks	document_id, chunk_id, content, vector, score	Векторний індекс FastAPI/LanceDB.

3.3.4 Підтримка цілісності даних

Цілісність даних підтримується на рівні доменних методів. `UniversityDocument` не створюється напряду через публічний конструктор, а має фабричний метод `Create`. Цей метод перевіряє, що назва, вміст, посилання, `GoogleDriveId` і `ContentHash` не порожні. Також перевіряється, що посилання містить `GoogleDriveId`. Це запобігає створенню некоректного документа.

Під час оновлення документа викликається метод `Update`. Він перевіряє ті самі інваріанти й після успішного оновлення переводить документ у стан `NotIndexed`. Це важливо, бо зміна тексту або `hash` автоматично означає, що старі `embeddings` можуть бути неактуальними.

Після успішної індексації викликається `MarkAsIndexed`, який встановлює статус `Indexed`, дату індексації та очищує помилку. Якщо індексація завершується помилкою, викликається `IndexFailed`, який зберігає текст помилки. Така модель дозволяє діагностувати проблеми й повторювати індексацію.

На рівні API вхідні дані перевіряються `FluentValidation`. `Surface-level validation` відокремлена від доменних інваріантів. Наприклад, `validator` перевіряє, що `Query` не порожній, а доменна сутність перевіряє власну коректність. Це відповідає принципу розподілу відповідальності.

3.3.5 Модель графового подання результатів пошуку

Графове подання в `EduGraph` є `runtime`-структурою клієнтського інтерфейсу, а не окремою графовою базою даних. Воно формується у `React` після завантаження папок `Google Drive` та після отримання результатів семантичного пошуку. Такий підхід достатній для задачі навігації: користувач бачить, до яких розділів корпусу належать знайдені документи, але система не вводить окрему онтологічну модель предметної області.

Таблиця 3.1 – Структура вузлів графового подання

Тип вузла	Джерело даних	Основні поля	Призначення
root	Константа frontend	id, title	Початкова точка графа, що відповідає корпусу G7
trunk	Алгоритм побудови layout	id, parentId	Технічний вузол для акуратного розгалуження структури
folder	Endpoint /google-drive/folders	id, title, url, parentId	Відображення папки Google Drive і перехід до неї
document	Endpoint /google-drive/search-documents	title, url, content, folderName, chunkId	Результат пошуку з релевантним фрагментом

Зв'язок документа з папкою в інтерфейсі виконується за значенням folderName. Якщо відповідна папка знайдена в уже завантаженій структурі, вузол документа приєднується до неї; якщо ні, документ приєднується до root-вузла. Це запобігає втраті результату навіть тоді, коли назва папки в індексі не збігається зі списком актуальних папок.

Структура графового подання результатів пошуку EduGraph

Runtime-граф формується у frontend на основі папок Google Drive та результатів семантичного пошуку.

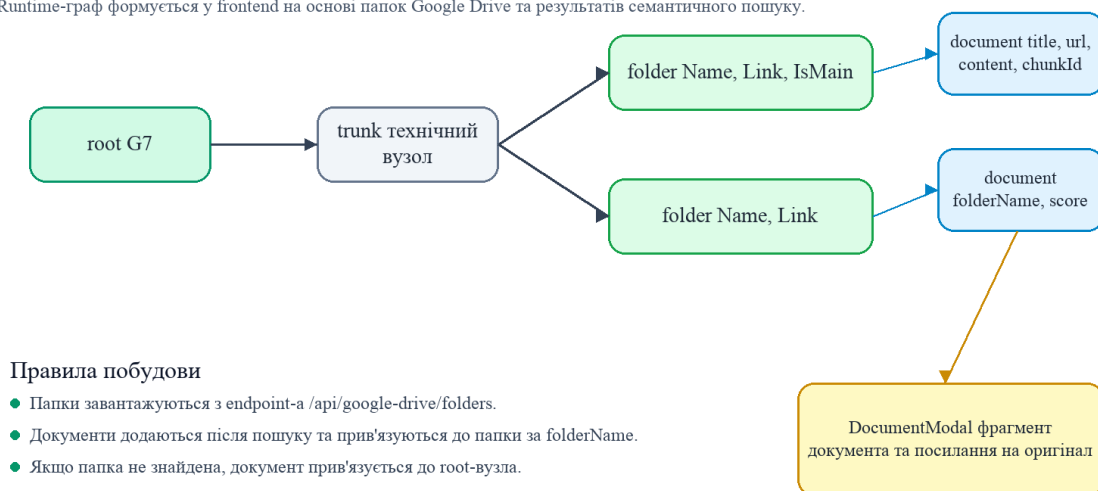


Рисунок 3.2 – Структура графового подання результатів пошуку EduGraph

3.4 Проектування серверної частини застосунку

Серверна частина EduGraph побудована на ASP.NET Core Minimal API. На відміну від класичної структури з контролерами, у проєкті використано вертикальні функціональні зрізи. Кожна функція описує власний request, validator, endpoint і handler. Наприклад, login, signup, пошук документів або підтвердження заявки можуть розглядатися як окремі зрізи.

Перевага такого підходу полягає в локалізації логіки. Коли потрібно змінити конкретний endpoint, розробник відкриває один файл і бачить усі елементи сценарію. Це зменшує ризик випадкових змін у сусідній функціональності та відповідає принципу «touch only what you must».

Endpoint-и автоматично реєструються через інтерфейс IEndpoint. Це дозволяє не підтримувати великий централізований файл маршрутизації. Кожен endpoint сам описує, який route він додає, які policies застосовує, які response-и може повертати та які tags використовуються в документації.

Авторизація реалізована через JWT Bearer і policies. Після login користувач отримує token з role claim. Backend endpoint-и використовують

RequireAuthorization з потрібними політиками. Наприклад, пошук доступний будь-якій авторизованій ролі, а керування адміністраторами – тільки SuperAdmin.

Для Google Drive інтеграції створено окремий infrastructure-проект. GoogleDriveService відповідає за root folder link, список папок, рекурсивний обхід файлів, export Google Workspace документів і парсинг контенту. Це відокремлює зовнішній API від доменної логіки синхронізації.

GoogleDriveSyncService координує синхронізацію. Він отримує папки, оновлює UniversityFolders, stream-ить документи batch-ами, оновлює UniversityDocuments, визначає видалені документи й запускає індексацію змінених. Саме цей сервіс поєднує Google Drive, SQLite і vector-search.

VectorSearchService є HTTP-обгорткою над FastAPI. Він має методи SearchAsync, UpsertDocumentsAsync і DeleteDocumentsAsync. Це дозволяє core-коду не знати деталей адрес FastAPI endpoint-ів і працювати через Result pattern. У разі помилки сервіс повертає ErrorDetails, а не кидає виняток як частину бізнес-потoku.

Серверна частина також використовує фонові служби. Періодична синхронізація підтримує актуальність корпусу, а примусова синхронізація дозволяє користувачу з правами оновити індекс вручну. Це важливо для сценаріїв, коли документи щойно додані в Google Drive і повинні швидко потрапити в пошук.

3.4.1 Маршрути API та політики доступу

Маршрути API в EduGraph згруповані за функціональними областями. Auth відповідає за вхід і подання заявки. Students, Teachers і Admins відповідають за керування користувачами відповідних типів. SignUpApplications відповідає за життєвий цикл заявок. GoogleDrive відповідає за папки, root-folder link, синхронізацію та пошук документів. Така структура зрозуміла як для розробника, так і для опису в документації.

Кожен endpoint має власну політику доступу. Пошук документів дозволений усім авторизованим ролям, тому він використовує політику

AnyRole. Адміністративні endpoint-и мають жорсткіші обмеження. Наприклад, керування адміністраторами доступне тільки суперадміністратору, тоді як керування студентами й викладачами може бути доступне викладачу, адміністратору або суперадміністратору залежно від бізнес-правил.

Політики доступу важливі не тільки для безпеки, а й для структури frontend. Якщо backend дозволяє дію тільки певним ролям, frontend повинен не показувати відповідний маршрут іншим користувачам. Проте frontend-перевірка не є достатньою сама по собі. Остаточне рішення завжди має приймати backend, оскільки клієнтський код може бути змінений користувачем.

У Minimal API endpoint описує route, handler, authorization, response types і tags. Це робить endpoint самодостатнім. Наприклад, endpoint пошуку документів приймає Request з Query, перевіряє validator, викликає VectorSearchService і повертає колекцію Response. Така локальна структура полегшує тестування та читання коду.

Валідація вхідних даних виконується через FluentValidation. Для пошуку перевіряється, що Query не порожній. Для форм реєстрації або створення користувача перевіряються логін, пароль, тип користувача та інші поля. Валідація API-рівня не замінює доменні інваріанти, а виконує попередню перевірку поверхневих помилок.

Обробка помилок побудована через Result pattern і перетворення помилок у HTTP-відповіді. Такий підхід дозволяє не використовувати винятки для звичайних бізнес-ситуацій, наприклад неправильного логіна, невалідної заявки або помилки зовнішнього сервісу. Для пояснювальної записки це важливо, бо показує керовану модель помилок.

API також виконує роль стабільного контракту між frontend і внутрішньою інфраструктурою. Навіть якщо у майбутньому FastAPI буде замінено іншим сервісом або LanceDB – іншим vector store, клієнтський endpoint search-documents може залишитися незмінним. Це знижує зв'язність між UI та ML-компонентом.

3.4.2 Реєстрація, заявки та створення користувачів

Реєстрація через заявки є важливим рішенням для системи EduGraph. У відкритому веб-застосунку користувач часто може створити обліковий запис одразу, але для освітнього корпусу це не завжди прийнятно. Документи можуть бути внутрішніми, методичними або призначеними для певної групи користувачів. Тому створення облікового запису має бути контрольованим.

Користувач подає заявку через frontend. Заявка містить дані, потрібні для майбутнього облікового запису, але сама по собі не дає доступу. Backend зберігає її як `SignUpApplication` зі статусом очікування. Такий стан дозволяє адміністративній ролі переглянути заявку та прийняти рішення.

Підтвердження заявки має створити користувача та призначити йому роль. Цей процес повинен бути атомарним з погляду бізнес-логіки: не можна допустити ситуацію, коли користувач створений, але роль не призначена, або заявка позначена як підтверджена, але обліковий запис не існує. Тому обробка заявки повинна виконуватися послідовно й із перевіркою результатів кожного кроку.

Відхилення заявки є окремим сценарієм. Воно не повинно видаляти історію заявки, оскільки для адміністратора може бути важливо бачити, що певна особа вже зверталася. Статус заявки дозволяє відокремити активні заявки від оброблених і уникнути повторної обробки.

Для студентів додатковим полем може бути навчальна група. Це корисно для ідентифікації користувача в університетському контексті. Для викладачів і адміністраторів група може бути не потрібна, тому модель має враховувати різні типи користувачів без дублювання сутностей.

На frontend заявки відображаються у вигляді списку. Користувач з правами обробки бачить дані заявки та кнопки дій. Важливо, щоб UI чітко показував статус і не дозволяв виконувати дію, яка вже неактуальна. Наприклад, підтверджену заявку не потрібно повторно підтверджувати.

Така модель реєстрації відповідає принципу контрольованого доступу. Вона додає один організаційний крок, але підвищує безпеку й керованість

системи. Для дипломної роботи це важливо, оскільки EduGraph розглядається не як публічний пошуковик, а як внутрішня освітня система.

3.4.3 Алгоритм синхронізації Google Drive

Синхронізація Google Drive є одним з центральних процесів EduGraph. Її призначення полягає в тому, щоб перетворити зовнішню файлову структуру на локальний корпус, придатний для пошуку. При цьому система не повинна змінювати оригінальні файли. Вона тільки читає папки, документи, метадані та посилання.

Процес починається з отримання кореневої папки. У конфігурації зберігається `DefaultFolderId`, який визначає, з якої папки потрібно починати обхід. Сервіс отримує `webViewLink` кореня й за потреби створює в локальній базі службову папку `G7` з ознакою `IsMain`. Це дозволяє frontend мати центральний вузол графу.

Далі система отримує папки верхнього рівня та оновлює `UniversityFolders`. Якщо папка вже існує, оновлюються її назва та посилання. Якщо папка нова, створюється нова сутність. Якщо папка зникла з Google Drive, вона видаляється з локальної бази, якщо не є головною. Таким чином, структура папок у SQLite підтримується актуальною.

Після оновлення папок сервіс переходить до документів. Обхід виконується рекурсивно через чергу папок. Для кожної папки запитується список елементів. Якщо елемент є папкою, він додається до черги. Якщо елемент є файлом, його `id` додається до `batch` для завантаження. Такий алгоритм дозволяє пройти дерево без рекурсивного виклику методу й контролювати `batch`-розмір.

Для зменшення навантаження документи обробляються `batch`-ами. У коді використано `BatchSize 15` і `semaphore` з лімітом 15. Це практичне рішення, яке обмежує паралельні запити до Google Drive API. Без такого обмеження велика кількість файлів могла б призвести до помилок, `throttling` або нестабільної роботи.

Після отримання документа система витягує текст. Для Google Workspace документів виконується export у відповідний формат. Для DOCX використовується OpenXML, для PDF – PdfPig, для TXT і CSV – читання текстового потоку. Якщо після парсингу текст порожній, документ не може бути корисно проіндексований, тому повертається помилка.

Коли batch документів отримано, GoogleDriveSyncService порівнює їх з існуючими UniversityDocuments. Для нових документів викликається Create. Для існуючих – Update. У кожному випадку обчислюється SHA-256 hash вмісту. Якщо документ змінюється, він позначається як NotIndexed, що запускає подальшу індексацію.

Окремо обробляються видалені документи. Після завершення обходу система має множину syncedIds. Усі локальні документи, GoogleDriveId яких не входить у syncedIds, вважаються видаленими з Google Drive. Для них виконується DeleteDocumentsAsync у vector-search сервісі, а потім ExecuteDeleteAsync у SQLite.

Такий алгоритм забезпечує узгодженість між Google Drive, локальною БД і vector index. Якщо документ додано, він потрапляє в SQLite і LanceDB. Якщо документ змінено, він переіндексується. Якщо документ видалено, він зникає з обох локальних сховищ. Це робить пошуковий корпус актуальним.

3.4.4 Парсинг документів і обмеження форматів

Парсинг документів є критичним етапом, оскільки semantic search працює з текстом, а не з файлом як таким. Якщо система не може отримати текст, вона не може створити embedding. Тому підтримка форматів прямо впливає на повноту пошукового корпусу.

Для DOCX використовується OpenXML-підхід. Сервіс відкриває WordprocessingDocument і читає InnerText основного body. Це дозволяє отримати текст з більшості звичайних Word-документів. Обмеження полягає в тому, що складна верстка, таблиці або текст у нестандартних елементах можуть бути представлені не ідеально, але для semantic search часто достатньо основного текстового потоку.

Для PDF використовується PdfPig. Він читає сторінки й додає page.Text до загального рядка. Основне обмеження PDF полягає в тому, що не кожен PDF має текстовий шар. Якщо файл є сканованим зображенням, PdfPig не отримає текст без OCR. Це потрібно прямо зазначити в рекомендаціях щодо використання системи.

Для TXT і CSV використовується читання через StreamReader з UTF-8 і detectEncodingFromByteOrderMarks. Такі формати простіші, але можуть мати проблеми з кодуванням або структурою. CSV, наприклад, може містити таблицю, де зміст розподілений по колонках. Для базового пошуку достатньо прочитати текст, але для точнішої інтерпретації в майбутньому можна додати спеціальну обробку таблиць.

Google Workspace файли потребують export. Документ експортується в DOCX, spreadsheet – у CSV, presentation – у PPTX. Поточна реалізація парсить DOCX, PDF, TXT і CSV, тому презентації можуть потребувати окремого розширення парсера. У дипломній записці це варто описати як обмеження поточного прототипу.

Після парсингу текст не повинен одразу вважатися якісним. Він може містити зайві пробіли, розриви, повтори, службові фрагменти або погану структуру. У FastAPI-сервісі виконується базова нормалізація пробілів, але глибоке очищення тексту може бути напрямом подальшого розвитку. Наприклад, можна видаляти службові колонтитули або повторювані шаблонні частини.

Якість парсингу прямо впливає на embeddings. Якщо текст документа отриманий некоректно, vector search буде працювати з шумом. Тому підтримка форматів і якість витягування тексту є не допоміжною, а основною частиною системи семантичного пошуку.

3.4.5 Життєвий цикл індексації документа

Життєвий цикл індексації починається з появи або зміни документа в Google Drive. Під час синхронізації backend отримує текст і metadata, обчислює

ContentHash і створює або оновлює UniversityDocument. Якщо документ новий або змінений, його SearchIndexStatus встановлюється в NotIndexed.

Статус NotIndexed означає, що реляційна БД уже знає про документ, але vector index ще не оновлений. Це важливий проміжний стан. Він дозволяє не втратити документ навіть тоді, коли FastAPI тимчасово недоступний. Документ залишається в SQLite і може бути проіндексований пізніше.

IndexChangedDocumentsAsync вибирає документи зі статусом NotIndexed і непорожнім Content. Вибірка обмежена batchSize 15, що зменшує ризик надто великого HTTP-запиту до Python-сервісу. Документи перетворюються на VectorDocumentDto, де зберігаються id, title, content, url, contentHash і folderName.

FastAPI endpoint /documents/upsert для кожного документа спочатку видаляє старі chunk-и за document_id. Це робить upsert ідемпотентним з погляду індексу: після повторної індексації в LanceDB не повинні залишатися старі фрагменти того самого документа. Далі текст ділиться на chunk-и, для кожного chunk створюється embedding, і rows додаються до таблиці.

Після успішного upsert backend викликає MarkAsIndexed для кожного документа. У документі зберігається дата індексації, а помилка очищується. Це дозволяє бачити, коли документ востаннє був успішно проіндексований.

Якщо під час upsert сталася помилка, backend викликає IndexFailed і зберігає повідомлення. Такий документ не зникає, а переходить у стан Failed. У майбутньому можна додати адміністративний екран для перегляду failed документів і повторного запуску індексації.

Якщо документ видалено з Google Drive, його життєвий цикл завершується. Backend надсилає document_id до /documents/delete, Python-сервіс видаляє chunk-и, а SQLite-запис видаляється. Це потрібно для того, щоб пошук не повертав документи, яких більше немає в джерелі.

Життєвий цикл індексації показує, що EduGraph не просто виконує одноразовий import. Система підтримує актуальність корпусу, відстежує

зміни, має проміжні стани та може відновлюватися після помилок. Це відрізняє її від простого скрипта індексації.

3.5 Проектування клієнтської частини застосунку

Клієнтська частина EduGraph реалізована як React SPA. Вхідною точкою є `main.tsx`, а `App` підключає `AppRouter`. Маршрути централізовані в `AppRouter`, що дозволяє бачити повну карту навігації. Відкритими маршрутами є `login`, `signup` і `privacy`, а решта маршрутів захищена через `ProtectedRoute`.

`ProtectedRoute` перевіряє наявність ролі та дозволені ролі маршруту. Якщо користувач не авторизований, він переходить на `login`. Якщо роль не відповідає маршруту, користувач перенаправляється на маршрут за замовчуванням для своєї ролі. Це забезпечує узгодженість frontend-навігації з backend-політиками.

API-запити централізовані через `apiFetch`. Він додає `Content-Type`, `Bearer token`, обробляє 401 і нормалізує базову адресу API. Це важливо, бо інакше кожен api-модуль мав би дублювати логіку авторизації та обробки помилок. API-модулі розділені за доменами: `auth`, `users`, `students`, `teachers`, `admins`.

Сторінка `StudentSearchPage` є найскладнішою частиною frontend. Вона завантажує папки, зберігає `query`, керує `pageState`, відображає `loading overlay`, будує `graph data`, позиціонує вузли й відкриває `DocumentModal`. У цій сторінці поєднуються API, стан інтерфейсу, графова візуалізація та взаємодія з користувачем.

Граф будується з кореневого вузла `G7`, `trunk`-вузлів, папок і документів. Документи прив'язуються до папки за `folderName`. Якщо відповідна папка не знайдена, документ прив'язується до кореня. Це дозволяє не втрачати результат навіть тоді, коли метадані папки неповні або відрізняються від списку папок.

UI також містить адміністративні сторінки: списки студентів, викладачів, адміністраторів, форму додавання користувача та сторінку заявок. Такі сторінки використовують повторювані компоненти таблиць, пагінації та

полів. Це зменшує дублювання й робить адміністративний інтерфейс послідовним.

Для Google Drive передбачено окремий компонент controls. Він дозволяє відкрити кореневу папку та запустити синхронізацію. Посилання перевіряються через `safeUrl`, що зменшує ризик відкриття некоректного або небезпечного URL.

Стилізація побудована на Tailwind CSS, власних токенах і inline styles для складної графової частини. Для дипломної роботи важливо описати не тільки зовнішній вигляд, а й функціональний сенс інтерфейсу: користувач бачить стан системи, результати пошуку, контекст папок і фрагмент документа.

3.5.1 Навігація, маршрути та рольовий frontend

Frontend-навігація в EduGraph відображає рольову модель backend. AppRouter містить маршрути для login, signup, privacy, student/search, teacher/search, admin/search, sign-up applications, add-user, teachers і admins. Кожен захищений маршрут обгорнутий у ProtectedRoute з переліком дозволених ролей.

Розділення маршрутів за ролями робить інтерфейс зрозумілішим. Студент не бачить адміністративну логіку. Викладач отримує доступ до своїх робочих сценаріїв. Адміністратор має інший набір сторінок. Суперадміністратор отримує найширшу навігацію. Такий підхід дозволяє використовувати один SPA для всіх ролей без дублювання застосунку.

ProtectedRoute не тільки перевіряє, чи є роль, а й перенаправляє користувача відповідно до ситуації. Якщо користувач не авторизований, він іде на login. Якщо роль не підходить, він іде на маршрут за замовчуванням для своєї ролі. Це зменшує кількість «мертвих» сторінок і покращує користувацький досвід.

Роль читається з JWT через утиліти auth. У токени підтримується claim Microsoft role або простий role. Таке рішення враховує особливості .NET

Identity/JWT і робить frontend сумісним із backend-форматом токена. Також перевіряється `exp`, щоб не використовувати прострочений token.

API-клієнт `apiFetch` є центральним місцем HTTP-взаємодії. Він зменшує дублювання коду, додає авторизацію й уніфікує поведінку при 401. Без такого клієнта кожен API-модуль мав би власну логіку заголовків, token і помилок, що підвищило б ризик несумісності.

Рольова навігація повинна розглядатися як допоміжний рівень безпеки. Вона покращує UX, але не замінює backend authorization. Якщо користувач вручну змінить URL, backend все одно має перевірити політику. Саме подвійна модель – frontend route guard і backend policy – робить систему надійнішою.

3.5.2 Графова візуалізація результатів

Графова візуалізація є відмінною рисою EduGraph. Більшість пошукових систем показує результати списком, але список не відображає структуру джерельного корпусу. У EduGraph документи походять з папок Google Drive, тому важливо показати користувачеві, де саме знаходиться результат у структурі матеріалів.

Базовий граф будується ще до виконання пошуку. Він містить root-вузол G7, trunk-вузли та folder-вузли. Це дає користувачу уявлення про структуру корпусу. Після пошуку до графа додаються document-вузли, прив'язані до папок за `folderName`. Якщо `folderName` відсутній або не знайдений, документ приєднується до root.

Для візуалізації використано `react-force-graph-2d`. Однак стандартна поведінка `force graph` не завжди достатня для акуратного відображення документів і папок. Тому в `StudentSearchPage` реалізовано власну логіку layout: обчислення ширини вузлів, позиціонування папок, розміщення груп документів, адаптація під ширину екрана та скорочення довгих назв.

Окремі метрики графа визначені для desktop, tablet і mobile. Це важливо, бо документні назви можуть бути довгими, а кількість папок і результатів змінюється. Якщо не контролювати розміри вузлів і відстані, граф може стати

нечитабельним. Тому в коді є `GraphMetrics` з параметрами `documentNodeWidth`, `folderNodeHeight`, `branchGap`, `linkDistance` та іншими.

Вузол документа не є просто точкою. Він має назву, `url`, `content`, `folderName` і `parentId`. Саме `content` використовується для модального вікна. Таким чином, граф є не тільки візуальним елементом, а й інтерактивною структурою даних, яка веде до перегляду фрагмента.

Клік по папці відкриває Google Drive URL, а клік по документу відкриває `modal`. Це різні дії для різних типів вузлів. Така поведінка відповідає семантиці графа: папка є навігаційним контейнером, а документ є результатом пошуку.

Безпечність посилань забезпечується через `safeUrl`. Це важливо, оскільки URL надходить із зовнішнього джерела. Навіть якщо Google Drive є довіреним джерелом, `frontend` не повинен без перевірки відкривати довільні рядки як посилання.

Графова візуалізація має аналітичну цінність. Якщо багато результатів зосереджено біля однієї папки, користувач бачить тематичний центр відповіді. Якщо результати розкидані, це може свідчити, що запит має міждисциплінарний характер або що корпус містить дублікати тем у різних розділах.

3.5.3 Обробка станів і помилок у клієнтському інтерфейсі

Стан сторінки пошуку не можна описати одним `boolean loading`. У `EduGraph` є кілька станів: завантаження папок, готовність до пошуку, виконання пошуку, наявність результатів, порожній результат і помилка. Кожен стан потребує окремого відображення, щоб користувач розумів, що відбувається.

Під час завантаження папок користувач не може повноцінно працювати з графом, тому інтерфейс повинен показувати `loading`. Якщо папки завантажилися, але пошук ще не виконано, граф може показувати базову структуру. Якщо пошук виконується, `loading overlay` пояснює, що система обробляє запит.

Порожній результат не є технічною помилкою. Це коректний сценарій, коли система не знайшла фрагментів, які проходять пороги score або rerank. UI повинен відрізняти порожній результат від помилки сервера. У першому випадку користувачу можна запропонувати змінити запит, у другому – повідомити про проблему.

Помилка може виникнути на різних рівнях: API недоступний, JWT прострочений, vector-search повернув 500, Google Drive sync ще не завершена або frontend не зміг побудувати граф. Централізований apiFetch допомагає обробляти частину таких помилок, але сторінка все одно повинна мати власний state error.

Для пошукового інтерфейсу важливо не приховувати очікування. Семантичний пошук може тривати довше, ніж звичайний список, бо включає embedding, vector search і reranking. Loading steps роблять очікування зрозумілішим і показують, що система працює, а не зависла.

Стан selectedDocument потрібний для модального вікна. Якщо користувач натискає на документ, state зберігає вузол, а modal читає його content і url. Закриття modal очищує selectedDocument. Це простий, але важливий механізм взаємодії.

3.6 Опис основних класів, методів і змінних

Основним класом інтеграції з Google Drive є GoogleDriveService. Він має методи для отримання root folder link, списку папок і stream-обробки документів. Сервіс використовує DriveService, read-only доступ, batch-обробку та semaphore для обмеження кількості паралельних запитів. Це важливо, оскільки велика кількість файлів може створити надмірне навантаження на зовнішній API.

GoogleDriveSyncService є центральним сервісом синхронізації. Його метод SynchronizeAsync об'єднує кілька задач: отримання папок, оновлення UniversityFolders, отримання документів, оновлення UniversityDocuments, видалення зниклих файлів і запуск IndexChangedDocumentsAsync. У межах

дипломної роботи цей сервіс можна розглядати як основний бізнес-процес системи.

UniversityDocument є ключовою доменною сутністю документного корпусу. Вона містить поля FolderName, Name, Content, Link, GoogleDriveId, ContentHash, SearchIndexStatus, SearchIndexedAt і SearchIndexError. Методи Create, Update, MarkAsIndexed, IndexFailed і MarkAsNotIndexed реалізують життєвий цикл документа щодо індексації.

VectorSearchService відповідає за HTTP-зв'язок з FastAPI. Метод SearchAsync надсилає запит search, UpsertDocumentsAsync надсилає документи на /documents/upsert, а DeleteDocumentsAsync видаляє chunk-и через /documents/delete. Цей клас приховує деталі HTTP-взаємодії від core-сервісів.

JwtService відповідає за генерацію токена. Він додає user id, nickname, jti та role claim. На frontend роль читається з токена й використовується для маршрутизації. Таким чином, JWT є зв'язком між backend-авторизацією та frontend-навігацією.

У FastAPI-сервісі важливими моделями є IncomingDocument, UpsertRequest, DeleteRequest, SearchRequest і SearchResult. IncomingDocument містить id, title, content, url, content_hash і folder_name. SearchRequest містить query, top_k, min_score і min_rerank_score. SearchResult повертає document_id, chunk_id, chunk_index, title, content, url, folder_name, score і rerank_score.

Функція chunk_text нормалізує пробіли та ділить текст через RecursiveCharacterTextSplitter. get_or_create_table створює таблицю LanceDB, якщо вона ще не існує. Endpoint /documents/upsert видаляє старі chunk-и документа, створює нові embeddings і додає рядки. Endpoint /search виконує query embedding, vector search, reranking і повертає top_k результатів.

У frontend ключовою функцією є buildGraph. Вона створює root-вузол, folder-вузли, document-вузли та links. Також важливими є applyGraphLayout і допоміжні функції для вимірювання тексту, скорочення підписів, визначення ширини вузлів і розміщення документів біля відповідних папок.

3.6.1 Алгоритм семантичного пошуку у FastAPI-сервісі

Алгоритм semantic search у FastAPI складається з двох основних частин: індексації та пошуку. Індексція виконується через /documents/upsert, а пошук – через /search. Такий поділ відповідає типовій архітектурі retrieval-систем, де підготовка corpus index і виконання query є різними процесами.

Під час upsert сервіс отримує список IncomingDocument. Кожен документ має id, title, content, url, content_hash і folder_name. Сервіс перевіряє, чи content не порожній. Якщо текст порожній, документ пропускається. Це важливо, бо порожній текст не може бути корисно перетворений у embedding.

Перед додаванням нових chunk-ів сервіс видаляє старі chunk-и за document_id. Це дозволяє уникнути дублювання після повторної індексації. Якщо документ змінився, старі фрагменти більше не повинні брати участь у пошуку. Upsert повинен залишати в індексі тільки актуальну версію документа.

Текст ділиться через RecursiveCharacterTextSplitter з chunk_size 1000 і overlap 150. Overlap потрібний для того, щоб фрагменти на межі chunk-ів не втрачали контекст. Якщо речення або абзац розрізаний, overlap дає сусідньому chunk частину попереднього контексту.

Для embeddings використовується префікс passage: для фрагментів і query: для запиту. Це відповідає підходу моделей E5, де різні типи тексту позначаються різними префіксами. Нормалізація embeddings дозволяє використовувати cosine similarity для порівняння.

Під час пошуку сервіс обчислює query embedding і запитує LanceDB. Кількість кандидатів береться більшою за top_k, оскільки перший етап має знайти ширший набір потенційно релевантних фрагментів. Далі кандидати фільтруються за min_score. Це відсікає слабкі результати до reranking.

CrossEncoder отримує пари query і title plus content. На відміну від embedding similarity, CrossEncoder розглядає пару разом, тому може точніше оцінити релевантність. Недолік у тому, що CrossEncoder повільніший, тому його застосовують тільки до обмеженого набору кандидатів.

Після reranking результати сортуються за rerank_score. Сервіс залишає не більше одного chunk-а на документ, щоб видача не складалася з кількох фрагментів одного великого файла. Це рішення робить результати різноманітнішими й кориснішими для користувача.

Фінальний SearchResult містить document_id, chunk_id, chunk_index, title, content, url, folder_name, score і rerank_score. Backend не повертає frontend усі внутрішні деталі, але ключові поля title, content, url і folderName потрібні для графу й modal.

3.7 Структура інтерфейсу користувача

Інтерфейс користувача EduGraph має бути не декоративним, а робочим. Його завдання – швидко привести користувача від інформаційної потреби до релевантного документа. Тому першим важливим екраном є login. Він повинен бути простим, оскільки автентифікація є службовою дією, а не головною цінністю системи.

Сторінка signup потрібна для створення заявки. Вона повинна пояснювати користувачеві, що після подання заявки доступ не з'являється миттєво. Це організаційно важливо, бо система працює з освітніми матеріалами й повинна контролювати, хто отримує доступ.

Сторінка пошуку є центральним інтерфейсом. Вона містить поле запити, граф, стани завантаження й модальне вікно. Користувач не повинен бачити технічні деталі embeddings або reranking. Він повинен бачити зрозумілу дію: ввести запит, отримати результати, відкрити фрагмент і перейти до документа.

Графове подання має кілька типів вузлів. Кореневий вузол відповідає головному сховищу, папки – структурним розділам, документи – результатам пошуку. Такий поділ допомагає користувачу зрозуміти, чи належить результат до певного тематичного розділу.

Модальне вікно документа показує фрагмент, який був знайдений системою. Це підвищує довіру до пошуку, оскільки користувач бачить причину релевантності. Без фрагмента система перетворилася б на список посилань, де потрібно вручну відкривати кожен файл.

Адміністративний інтерфейс складається з таблиць. Для таких сторінок важливі чіткі колонки, пагінація, кнопки дій і відсутність зайвих декоративних елементів. Адміністратор виконує повторювані операції, тому інтерфейс має бути передбачуваним.

Навігація формується залежно від ролі. Студент бачить пошук. Викладач і адміністратор бачать додаткові розділи. Суперадміністратор бачить керування адміністраторами. Такий підхід зменшує ризик, що користувач побачить недоступну або непотрібну функцію.

Важливим елементом є стани помилок і завантаження. Синхронізація, пошук і завантаження папок можуть тривати певний час. Якщо UI не показує стан, користувач може вважати систему неробочою. Тому loading overlay і pageState є не другорядними деталями, а частиною користувацького досвіду.

На рисунках 3.3–3.6 наведено ключові стани інтерфейсу. Вони показують не лише зовнішній вигляд сторінок, а й логіку взаємодії: завантаження структури папок, появу документів після пошуку, перегляд релевантного фрагмента та відмінність робочого сценарію викладача від студентського.

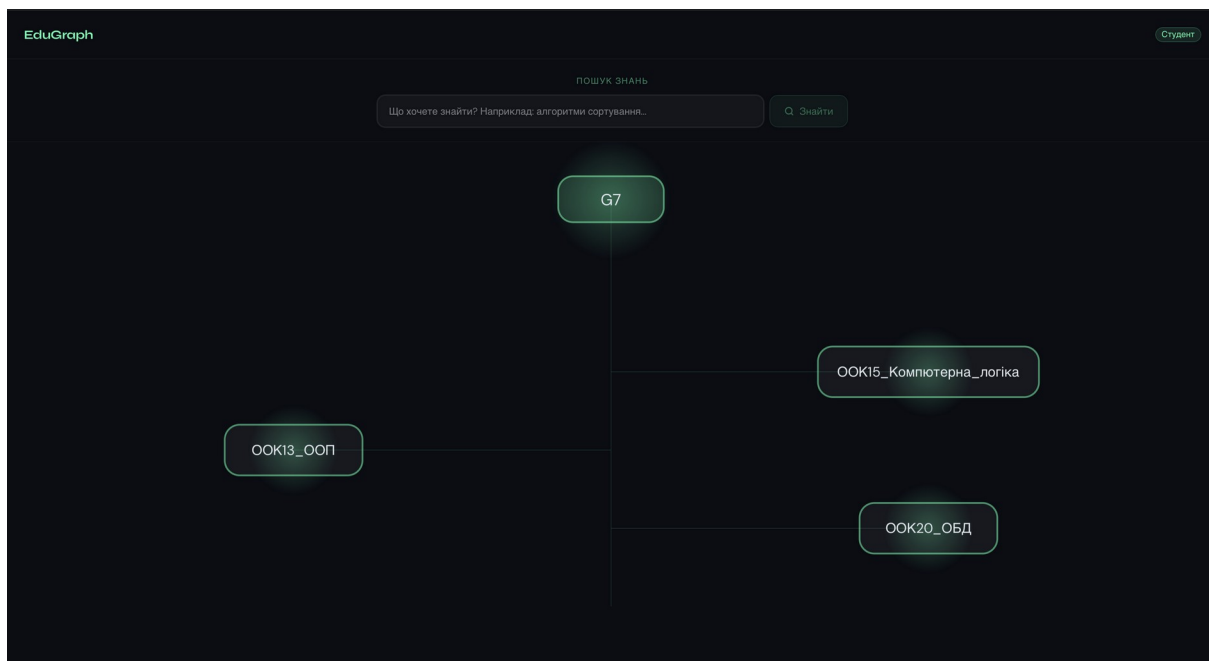


Рисунок 3.3 – Початковий стан графової навігації корпусом документів

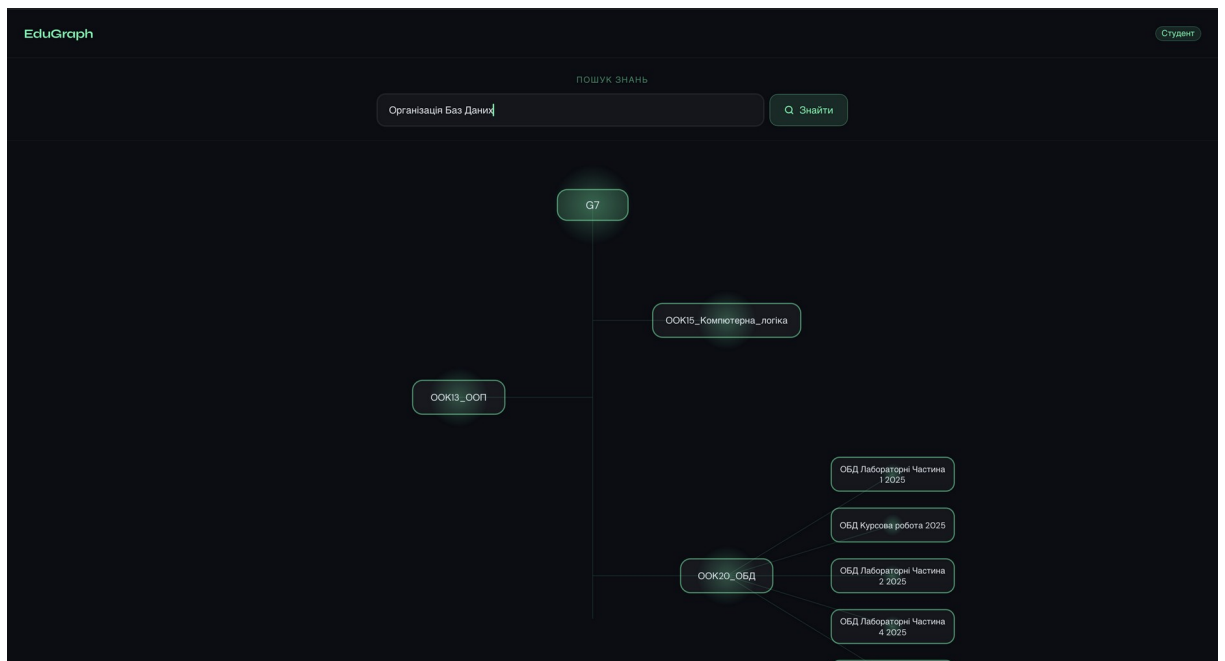


Рисунок 3.4 – Результати семантичного пошуку у вигляді графу

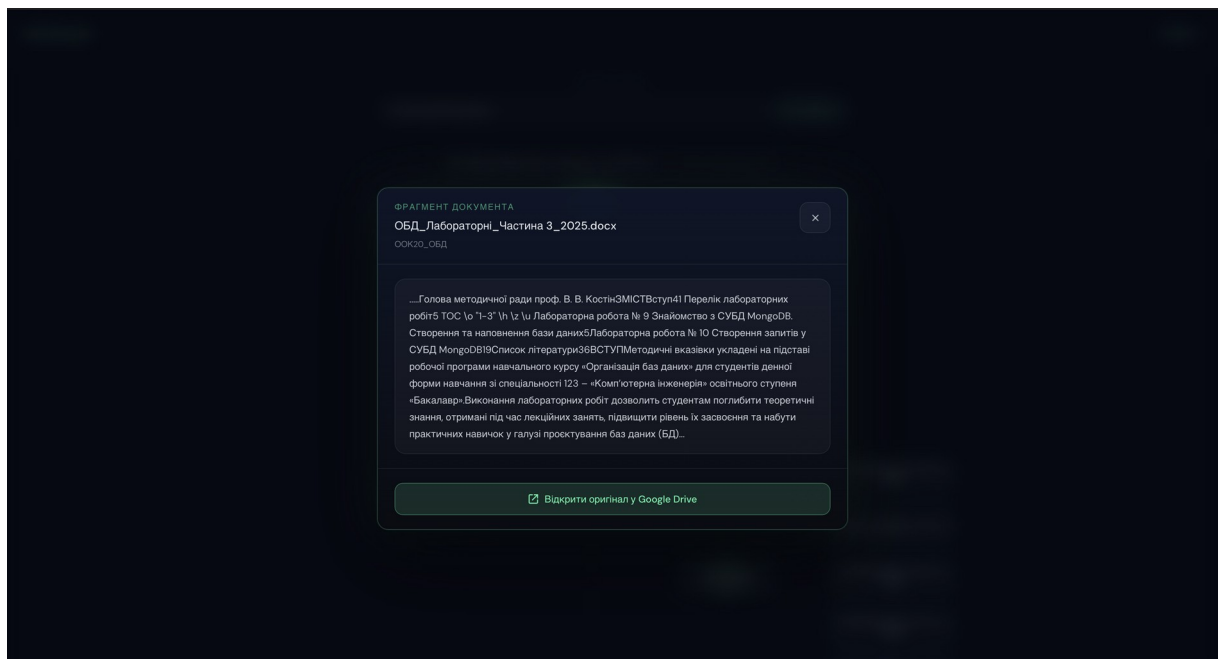


Рисунок 3.5 – Перегляд релевантного фрагмента документа

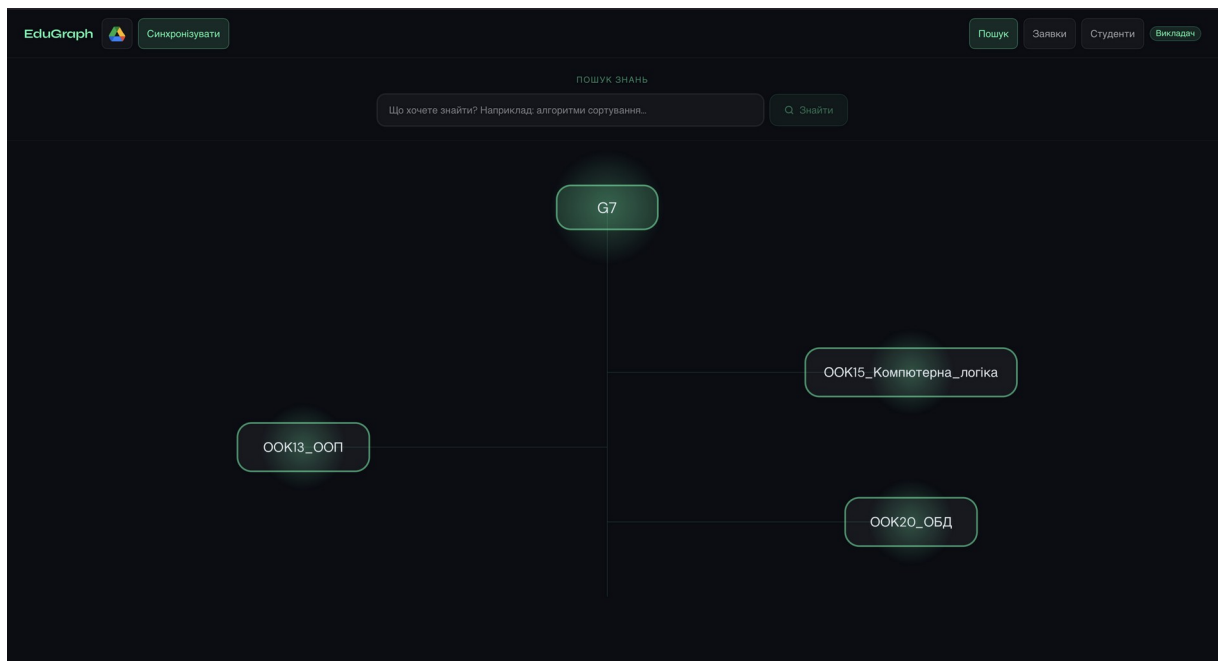


Рисунок 3.6 – Інтерфейс пошуку для користувача з роллю викладача

3.8 Експериментальне тестування спроектованого застосунку

Експериментальне тестування в цій роботі розглядається як функціональна перевірка основних сценаріїв, а не як повний benchmark інформаційного пошуку. Це відповідає поточним уточненням до роботи: кількісні метрики precision, recall, MRR або nDCG поки що не включаються. Водночас система повинна бути перевірена на рівні end-to-end сценаріїв.

Перший сценарій перевірки – вхід користувача. Потрібно перевірити, що коректний login і password повертають JWT, а неправильні дані не дають доступу. Також потрібно перевірити, що роль з токена правильно визначається frontend і користувач потрапляє на відповідний маршрут.

Другий сценарій – подання заявки. Користувач має заповнити форму, після чого заявка з'являється в адміністративному списку. Підтвердження заявки повинно створити обліковий запис, а відхилення – не створювати користувача. Важливо перевірити, що повторна обробка заявки не призводить до неконсистентного стану.

Третій сценарій – синхронізація Google Drive. Після запуску синхронізації система повинна отримати папки, документи, текстовий вміст і посилання. Для нових документів мають створюватися записи. Для змінених

документів має оновлюватися ContentHash і встановлюватися статус NotIndexed. Для видалених документів має виконуватися delete з vector index.

Четвертий сценарій – індексація. Документи зі статусом NotIndexed мають бути відправлені в FastAPI batch-ами. Після успішного upsert статус повинен змінитися на Indexed. Якщо FastAPI недоступний або повертає помилку, документ повинен отримати Failed, а система не повинна втратити інформацію про документ.

П'ятий сценарій – пошук. Користувач вводить запит, backend викликає vector-search, FastAPI повертає результати, frontend будує граф. Потрібно перевірити не тільки наявність відповіді, а й те, що кожен результат має title, url, content, score, chunkId і chunkIndex. Без цих полів граф і modal не зможуть працювати повноцінно.

Шостий сценарій – захист маршруту. Студент не повинен відкривати адміністративні сторінки. Користувач без токена не повинен відкривати пошук. Якщо токен недійсний, frontend має очистити авторизацію й перенаправити на login.

Сьомий сценарій – UI-перевірка графа. Потрібно переконатися, що граф не порожній після завантаження папок, що знайдені документи додаються до відповідних папок, що довгі назви скорочуються без руйнування layout, а клік по документу відкриває модальне вікно.

Восьмий сценарій – перевірка посилань. Google Drive URL повинні проходити через safeUrl, а відкриття оригіналу має вести до правильного документа. Це важливо, тому що система не зберігає повну копію джерела як офіційний документ, а спрямовує користувача до першоджерела.

3.8.1 Деталізована матриця функціональної перевірки

Деталізована перевірка системи повинна охоплювати не тільки окремі endpoint-и, а й повні сценарії. Для EduGraph важливо перевіряти зв'язок між компонентами, тому що багато помилок можуть виникати на межі frontend, backend і FastAPI. Наприклад, backend може повертати правильні дані, але frontend може неправильно прив'язати документ до папки.

Перевірка авторизації включає login, збереження token, читання role claim, доступ до маршрутів і реакцію на 401. Успішним результатом є ситуація, коли кожна роль бачить тільки свої сторінки, а backend відхиляє заборонені запити навіть при прямому HTTP-виклику.

Перевірка заявок включає створення заявки, перегляд pending заявок, approve і reject. Важливо перевірити, що approve створює користувача з правильною роллю, а reject не створює користувача. Також потрібно перевірити, що оброблена заявка не залишається у списку pending.

Перевірка синхронізації включає додавання нового документа в Google Drive, зміну документа та видалення документа. Для нового документа має з'явитися UniversityDocument і vector chunks. Для зміненого документа має змінитися ContentHash і відбутися upsert. Для видаленого документа має бути виконано delete в SQLite і LanceDB.

Перевірка пошуку включає запити різної довжини, порожній запит, запит без результатів і запит з очікуваними релевантними документами. Порожній запит повинен бути відхилений validator або повернути контрольований результат. Запит без результатів не повинен спричиняти помилку UI.

Перевірка графу включає завантаження папок, побудову root, folder і document вузлів, відкриття modal, відкриття Google Drive URL і адаптацію до різних розмірів екрана. Це особливо важливо, бо графова частина має власну логіку layout і може ламатися при довгих назвах документів.

Перевірка помилок включає недоступність FastAPI, помилку Google Drive, порожній текст документа й некоректний URL. У кожному випадку система має або зафіксувати помилку в статусі індексації, або показати користувачеві контрольований стан, а не завершитися непередбачуваним винятком.

Матриця перевірки повинна використовуватися як основа для майбутніх unit, integration і end-to-end тестів. У поточній версії записки вона описує ручне

та функціональне тестування, але надалі її можна перетворити на автоматизовані сценарії.

Група перевірки	Що перевіряється	Критерій успішності
Авторизація	Login, JWT, role claim, перенаправлення.	Користувач відкриває тільки дозволені сторінки.
Заявки	Створення, перегляд, approve, reject.	Заявка переходить у правильний статус без дублювання користувача.
Синхронізація	Папки, документи, hash, видалення.	SQLite і vector index відповідають Google Drive.
Пошук	Query, score, reranking, top_k.	Повертаються релевантні документи з фрагментами.
Граф	Побудова root, folders, documents, links.	Вузли не втрачаються, документ відкриває modal.

3.9 Інструкція користувача

Для початку роботи користувач відкриває веб-застосунок EduGraph у браузері. Якщо в нього вже є обліковий запис, він переходить на сторінку входу, вводить логін і пароль та натискає кнопку входу. Після успішної автентифікації система автоматично визначає роль користувача й відкриває відповідний розділ.

Якщо облікового запису немає, користувач переходить на сторінку реєстрації та подає заявку. У заявці потрібно вказати необхідні дані відповідно до форми. Після подання заявки користувач очікує рішення адміністратора або іншої уповноваженої особи. До підтвердження заявки доступ до закритих сторінок не надається.

Для виконання пошуку користувач відкриває сторінку пошуку. Система завантажує структуру папок і показує базовий граф. Користувач вводить запит

природною мовою. Запит може бути сформульований не як точна назва документа, а як змістова потреба, наприклад пошук фрагментів про компетентності, навчальні результати або певну тему.

Після запуску пошуку потрібно дочекатися завершення обробки. Якщо результати знайдені, у графі з'являються вузли документів. Користувач може візуально оцінити, до яких папок належать результати. Якщо результати відсутні, система показує відповідний стан, і користувач може змінити формулювання запиту.

Щоб переглянути знайдений фрагмент, потрібно натиснути на вузол документа. У модальному вікні відображається уривок тексту, назва документа й можливість перейти до оригіналу. Якщо користувач хоче перевірити повний контекст, він відкриває документ у Google Drive.

Користувач з адміністративними правами може перейти до сторінки заявок. На цій сторінці він переглядає заявки, підтверджує або відхиляє їх. Після підтвердження створюється користувач із відповідною роллю. Відхилена заявка не створює обліковий запис.

Адміністратор також може працювати зі списками користувачів. Він переглядає студентів, викладачів або адміністраторів залежно від власної ролі. Доступ до цих сторінок обмежений, тому звичайний студент не бачить і не відкриває такі маршрути.

Для оновлення корпусу документів уповноважений користувач може запустити синхронізацію Google Drive. Після цього система самостійно оновить папки, документи й індекс. Користувачу не потрібно вручну передавати файли до Python-сервісу або працювати з базою даних.

3.10 Узагальнені рекомендації щодо використання розробленого застосунку

Для ефективного використання EduGraph потрібно підтримувати впорядковану структуру Google Drive. Назви папок повинні бути змістовними, оскільки вони використовуються у графі. Якщо папки мають випадкові або технічні назви, графова навігація буде менш корисною для користувача.

Документи мають бути у форматах, які система може прочитати. DOCX, PDF, TXT і CSV підтримуються, однак скановані PDF без текстового шару можуть не дати придатного вмісту. Якщо корпус містить багато сканів, потрібно додати OCR-етап або підготувати текстові версії документів.

Синхронізацію доцільно запускати після значного оновлення корпусу. Якщо документи додаються часто, варто покладатися на фонову синхронізацію. Якщо потрібно швидко перевірити щойно додані матеріали, можна використовувати ручний запуск через інтерфейс.

Адміністративні права потрібно надавати обмежено. Оскільки адміністратор може впливати на користувачів і синхронізацію, така роль повинна призначатися тільки відповідальним особам. Суперадміністративний доступ має бути ще більш обмеженим.

Користувачам потрібно пояснити, що EduGraph не замінює Google Drive як офіційне сховище. Система допомагає знайти релевантний фрагмент і перейти до оригіналу. Остаточну перевірку змісту потрібно виконувати в повному документі.

Для подальшого розвитку рекомендовано додати фільтри за папками, типами документів і датою оновлення. Такі фільтри допоможуть звужити пошук, коли корпус стане більшим. Також корисним буде збереження історії запитів або останніх відкритих документів.

Окремим напрямом є оцінювання якості пошуку. Після підготовки еталонного набору запитів можна вимірювати precision, recall, MRR або nDCG. Це дозволить обґрунтовано порівнювати різні моделі embeddings, параметри chunking і пороги score.

У майбутньому можна додати генеративний LLM-шар. Однак він має спиратися на надійний retrieval. Спочатку система повинна знаходити правильні фрагменти, а вже потім генеративна модель може формувати узагальнену відповідь з посиланнями на джерела.

3.10.1 Обмеження поточної реалізації та напрями розвитку

Поточна реалізація EduGraph є прототипом, тому має обмеження. Перше обмеження стосується якості документного тексту. Якщо PDF є сканом без OCR або документ має складну структуру, система може отримати неповний або неякісний текст. Це впливає на embeddings і результати пошуку. Для майбутнього розвитку варто додати OCR або покращені парсери.

Друге обмеження стосується векторного сховища. LanceDB використовується локально через шлях `./vector_db`. Для демонстрації це зручно, але для production потрібно визначити, де саме зберігатиметься vector index, як він резервується, як переноситься між середовищами та як масштабується при збільшенні корпусу.

Третє обмеження пов'язане з метриками якості. Поки що система перевіряється функціонально, але не має benchmark-набору. Без контрольного набору запитів складно об'єктивно сказати, яка модель embeddings або який chunk_size кращі. Тому подальший розвиток має включати підготовку dataset для оцінювання.

Четверте обмеження стосується позицій фрагментів. Система повертає chunk_index і content, але не завжди знає сторінку або точну позицію в документі. Для користувача було б корисно відкривати не просто Google Drive файл, а місце всередині документа. Це складно через різні формати, але може значно покращити UX.

П'яте обмеження – відсутність генеративного LLM-шару. Тема містить LLM, але реалізована retrieval-основа. Це не є помилкою, якщо текст роботи правильно пояснює підхід. У майбутньому можна додати генерацію короткої відповіді на основі знайдених фрагментів, але тільки з обов'язковими посиланнями на джерела.

Шосте обмеження стосується frontend-фільтрів. Наразі основним способом уточнення є текстовий запит. Додаткові фільтри за папкою, типом документа або датою оновлення могли б покращити пошук у великому корпусі. Такі фільтри потребують додаткових полів у API та UI.

Сьоме обмеження – експлуатаційні вимоги. Для локального запуску достатньо поточних компонентів, але для реального впровадження потрібно визначити сервер, політику оновлення моделей, резервне копіювання SQLite і vector_db, моніторинг FastAPI та журналювання помилок синхронізації.

Незважаючи на ці обмеження, поточна архітектура створює правильну основу для розвитку. Вона вже розділяє frontend, backend і ML-сервіс, зберігає джерельні посилання, підтримує ролі та має життєвий цикл індексації. Тому подальші покращення можуть додаватися поступово, без повного переписування системи.

5 ЕКОНОМІЧНО-ОРГАНІЗАЦІЙНА ЧАСТИНА

Економічно-організаційна частина в цій роботі має розрахунково-обґрунтовальний характер і показує, як можна оцінити трудомісткість розроблення EduGraph. Оскільки точні ставки, нормативи й коефіцієнти залежать від методичних вимог кафедри, у записці подано структуру розрахунку та орієнтовний розподіл робіт.

Розроблення системи складається з аналітичних, проєктних, програмних, тестових і документаційних робіт. Аналітичний етап включає дослідження предметної області, типових рішень, ролей користувачів і вимог. Проєктний етап включає архітектуру, модель даних, API-контракти та сценарії взаємодії.

Програмний етап поділяється на backend, frontend і vector-search. Backend має найбільшу частку через авторизацію, ролі, заявки, Google Drive, фонову синхронізацію й інтеграцію з Python-сервісом. Frontend також потребує значного часу через графову візуалізацію та адміністративні сторінки. Vector-search вимагає роботи з ML-залежностями, embeddings, LanceDB і reranking.

Вид робіт	Трудомісткість, год	Результат
Аналіз предметної області	35	Вимоги, обмеження, постановка задачі.
Проектування архітектури	45	Компонентна схема, API, модель даних.
Backend	90	Авторизація, ролі, Google Drive, фонові синхронізація.
Vector-search	65	Chunking, embeddings, LanceDB, reranking.
Frontend	80	Маршрути, сторінки, граф, адміністративні сценарії.
Документація та	55	Пояснювальна

перевірка		записка, тестові сценарії, додатки.
-----------	--	-------------------------------------

Якщо прийняти сумарну трудомісткість 370 годин і умовну погодинну ставку S , прямі витрати на оплату праці можна оцінити як $C_p = 370 \times S$. До цієї суми можуть додаватися нарахування, витрати на обладнання, електроенергію, хмарні сервіси або підтримку інфраструктури. У поточному прототипі значна частина технологій є безкоштовною для локального використання, але production-розгортання може потребувати додаткових витрат.

Очікуваний економічний ефект полягає у скороченні часу пошуку документів. Якщо студент або викладач витрачає менше часу на пошук потрібного матеріалу, підвищується продуктивність навчальної та організаційної роботи. Навіть якщо система не має прямої комерційної монетизації, вона може давати організаційний ефект через зменшення непродуктивних витрат часу.

Додатковий ефект полягає у збереженні актуальності корпусу. Оскільки EduGraph працює з Google Drive як джерелом, користувачі переходять до оригіналів, а не до випадкових копій. Це зменшує ризик використання застарілих документів і полегшує супровід навчальних матеріалів.

Організаційні ризики пов'язані з якістю документів, доступністю Google Drive API, ресурсами для ML-моделей і політикою доступу. Якщо документи не мають текстового шару або мають погану структуру, якість пошуку знижується. Якщо сервер має недостатньо ресурсів, індексація може бути повільною. Якщо ролі налаштовані неправильно, користувачі можуть отримати надмірний або недостатній доступ.

Для зменшення ризиків потрібно підтримувати порядок у Google Drive, контролювати заявки, регулярно перевіряти статуси індексації, фіксувати помилки й планувати ресурси для vector-search сервісу. У майбутньому також

доцільно підготувати регламент оновлення моделей і повторної індексації корпусу.

ВИСНОВКИ

У кваліфікаційній роботі розглянуто розроблення системи EduGraph – веб-застосунку для семантичного пошуку в корпусі освітніх документів на основі LLM-орієнтованих transformer-моделей з інтерактивною графовою навігацією результатів. Робота розширює матеріали переддипломної практики та перетворює їх на повну пояснювальну записку з аналітичною, функціональною, проєктною та економічно-організаційною частинами.

У процесі аналізу предметної області встановлено, що традиційний пошук за точними словами не завжди відповідає потребам користувачів освітнього корпусу. Користувач часто шукає зміст, а не назву файла. Тому доцільним є використання semantic retrieval, де запит і фрагменти документів подаються у вигляді embeddings.

Було обґрунтовано архітектуру системи, що складається з React-клієнта, ASP.NET Core Web API, SQLite, Google Drive інтеграції та FastAPI-сервісу vector search. Такий поділ дозволяє відокремити користувацький інтерфейс, доменну логіку, збереження метаданих і ML/NLP-компонент.

У функціональній частині визначено ролі користувачів, варіанти використання, сценарії автентифікації, реєстрації через заявки, синхронізації Google Drive, індексації документів, семантичного пошуку та графового перегляду результатів. Окремо підкреслено, що frontend не звертається напряму до vector-search сервісу, а працює через backend API.

У проєктній частині описано вибір технологій, функціональні залежності, логічну й фізичну модель даних, цілісність даних, серверну частину, клієнтську частину, основні класи й методи, структуру інтерфейсу та тестові сценарії. Особливу увагу приділено UniversityDocument, GoogleDriveSyncService, VectorSearchService, FastAPI endpoint-ам і StudentSearchPage.

Практична реалізація показує, що EduGraph може синхронізувати документи з Google Drive, витягувати текст із підтримуваних форматів, зберігати метадані, індексувати фрагменти в LanceDB, виконувати пошук

через embeddings і reranking, а також відображати результати у графовій формі. Користувач бачить не тільки назву документа, а й релевантний фрагмент і посилання на першоджерело.

Кількісна оцінка якості пошуку в цій версії роботи не виконується, оскільки для неї потрібен окремий контрольний набір запитів і еталонних результатів. Проте в роботі визначено функціональні сценарії перевірки, які дозволяють оцінити працездатність основних компонентів системи.

Подальший розвиток системи доцільно спрямувати на додавання фільтрів, збереження координат фрагментів у документах, production-розгортання vector-search сховища, підготовку benchmark-набору для оцінювання якості пошуку та можливе підключення генеративного LLM-шару для формування відповідей із посиланнями на знайдені джерела.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Вихідний код проєкту EduGraph: frontend/edugraph.client, backend/api, backend/vector-search.
2. Методичні вказівки до виконання кваліфікаційної роботи бакалавра зі спеціальності 123 «Комп'ютерна інженерія». Версія 2025.
3. ДСТУ 8302:2015. Інформація та документація. Бібліографічне посилання. Загальні положення та правила складання.
4. Microsoft Learn. Minimal APIs overview. URL: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/minimal-apis>
5. Microsoft Learn. Entity Framework Core documentation. URL: <https://learn.microsoft.com/en-us/ef/core/>
6. Microsoft Learn. ASP.NET Core Identity. URL: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity>
7. React Documentation. URL: <https://react.dev/>
8. Vite Documentation. URL: <https://vite.dev/>
9. FastAPI Documentation. URL: <https://fastapi.tiangolo.com/>
10. Google for Developers. Google Drive API Documentation. URL: <https://developers.google.com/drive/api>
11. LanceDB Documentation. Vector Search. URL: <https://docs.lancedb.com/search/vector-search>
12. Sentence Transformers Documentation. Semantic Search. URL: https://www.sbert.net/examples/sentence_transformer/applications/semantic-search/README.html
13. Manning C. D., Raghavan P., Schütze H. Introduction to Information Retrieval. Cambridge University Press, 2008. URL: <https://nlp.stanford.edu/IR-book/>
14. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP, 2019. URL: <https://arxiv.org/abs/1908.10084>

15. Wang L. et al. Text Embeddings by Weakly-Supervised Contrastive Pre-training. 2022. URL: <https://arxiv.org/abs/2212.03533>
16. Nogueira R., Cho K. Passage Re-ranking with BERT. 2019. URL: <https://arxiv.org/abs/1901.04085>
17. OpenAPI Specification. URL: <https://spec.openapis.org/oas/latest.html>
18. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT). RFC 7519. URL: <https://www.rfc-editor.org/rfc/rfc7519>

ДОДАТКИ

Додаток А. Альбом графічного матеріалу

Додаток А містить графічні матеріали, які узагальнюють основні рішення пояснювальної записки: архітектуру системи, конвеєр синхронізації та пошуку, варіанти використання, послідовність виконання семантичного пошуку, логічну модель даних і сценарні екрани інтерфейсу EduGraph.

А.1 Узагальнена архітектура системи EduGraph

Схема А.1 використовується як графічне доповнення до відповідного розділу пояснювальної записки. Вона відображає не програмний код, а логіку побудови системи, тому може бути перенесена до альбому графічного матеріалу у форматі, який вимагається кафедрою.

Узагальнена архітектура системи EduGraph

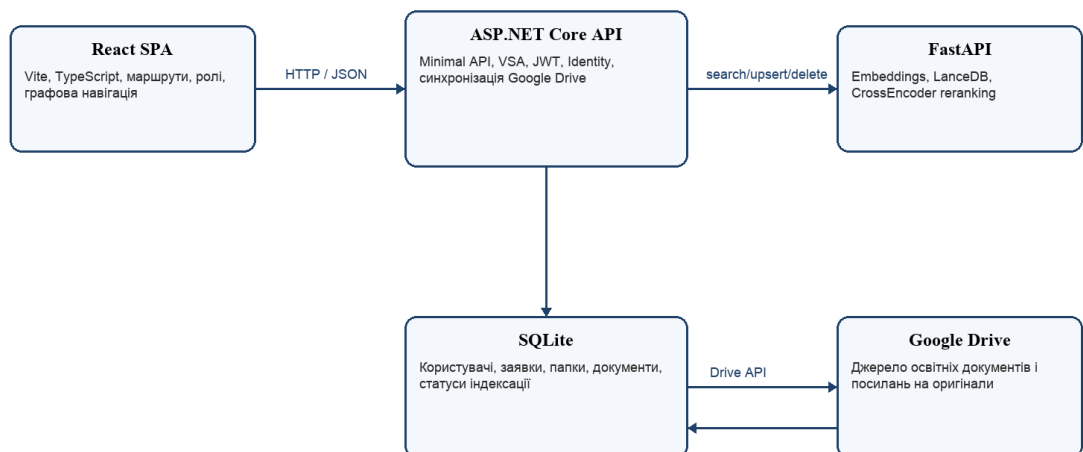


Рисунок А.1 – Узагальнена архітектура системи EduGraph

А.2 Конвеєр синхронізації, індексації та пошуку

Схема А.2 використовується як графічне доповнення до відповідного розділу пояснювальної записки. Вона відображає не програмний код, а логіку побудови системи, тому може бути перенесена до альбому графічного матеріалу у форматі, який вимагається кафедрою.

Конвеєр синхронізації, індексації та пошуку

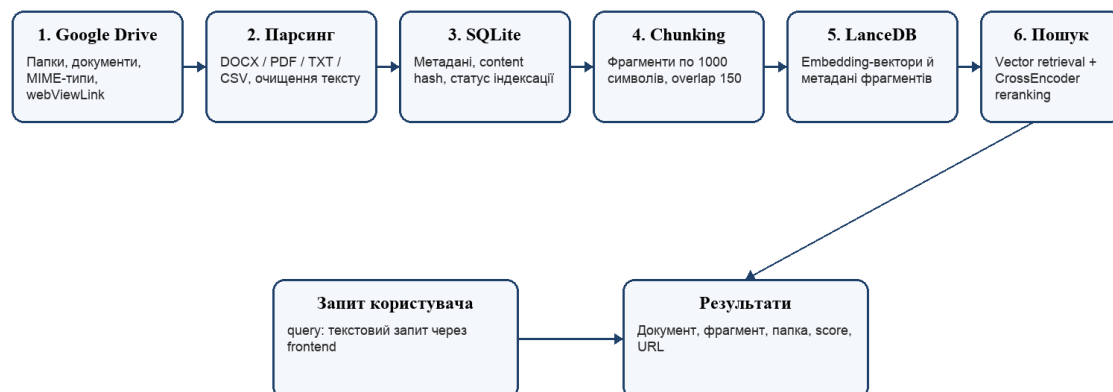


Рисунок А.2 – Конвеєр синхронізації, індексації та пошуку

А.3 Ролі користувачів та основні варіанти використання

Схема А.3 використовується як графічне доповнення до відповідного розділу пояснювальної записки. Вона відображає не програмний код, а логіку побудови системи, тому може бути перенесена до альбому графічного матеріалу у форматі, який вимагається кафедрою.

Ролі користувачів та основні варіанти використання

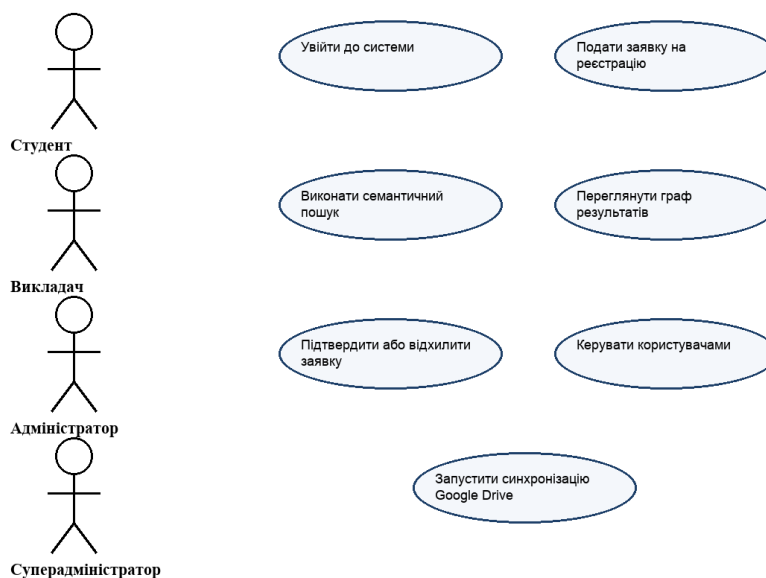


Рисунок А.3 – Ролі користувачів та основні варіанти використання

А.4 Послідовність виконання семантичного пошуку

Схема А.4 використовується як графічне доповнення до відповідного розділу пояснювальної записки. Вона відображає не програмний код, а логіку побудови системи, тому може бути перенесена до альбому графічного матеріалу у форматі, який вимагається кафедрою.

Послідовність виконання семантичного пошуку

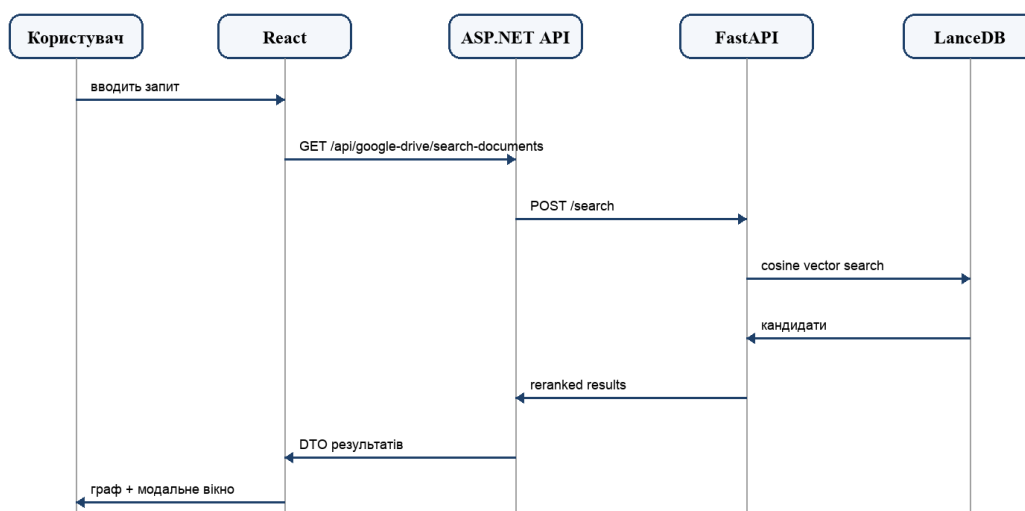


Рисунок А.4 – Послідовність виконання семантичного пошуку

А.5 Логічна модель даних EduGraph

Схема А.5 використовується як графічне доповнення до відповідного розділу пояснювальної записки. Вона відображає не програмний код, а логіку побудови системи, тому може бути перенесена до альбому графічного матеріалу у форматі, який вимагається кафедрою.

Логічна модель даних EduGraph

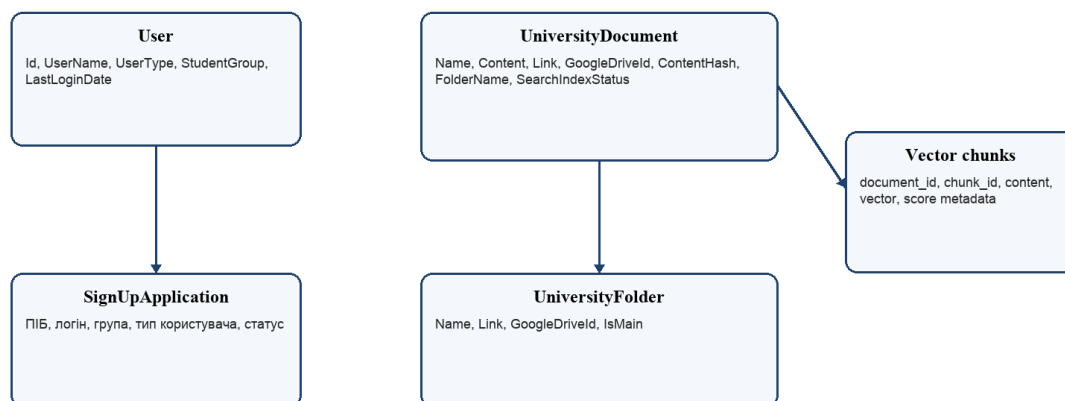


Рисунок А.5 – Логічна модель даних EduGraph

А.6 Сценарні екрани інтерфейсу EduGraph

Сценарні екрани доповнюють основні схеми та показують реалізацію рольової моделі у користувацькому інтерфейсі. Основний текст містить лише ключові стани, а в додатку наведено адміністративні сторінки, які підтверджують наявність окремих сценаріїв для викладача, адміністратора та суперадміністратора.

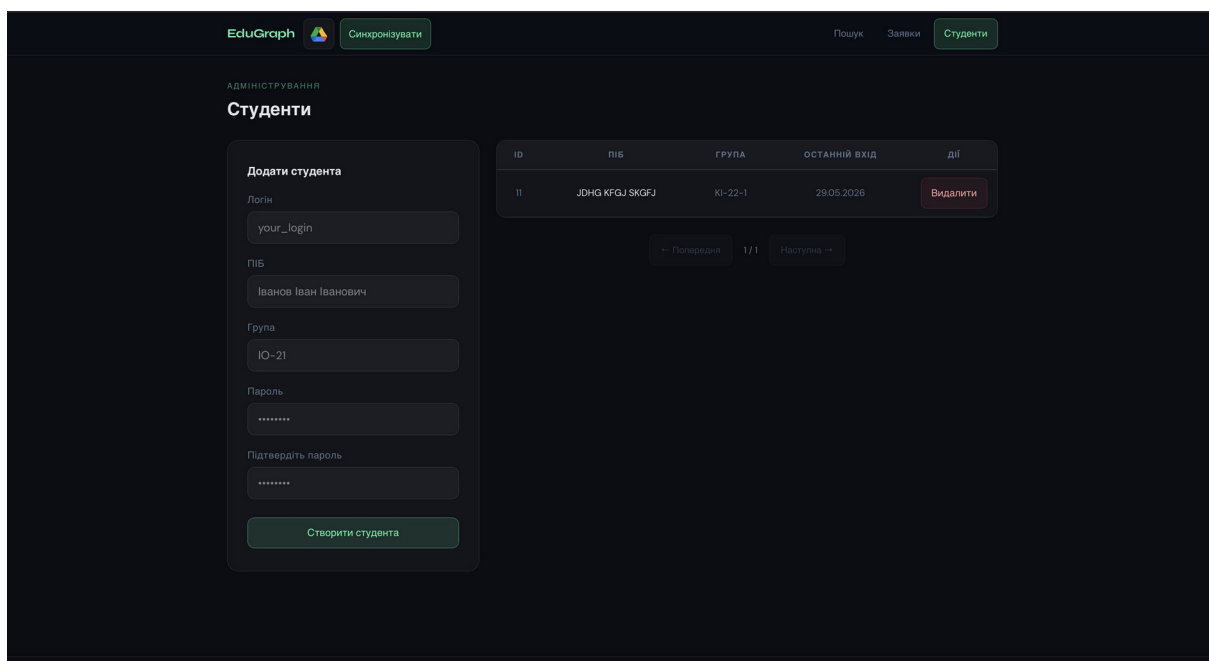


Рисунок А.6 – Сторінка керування студентами

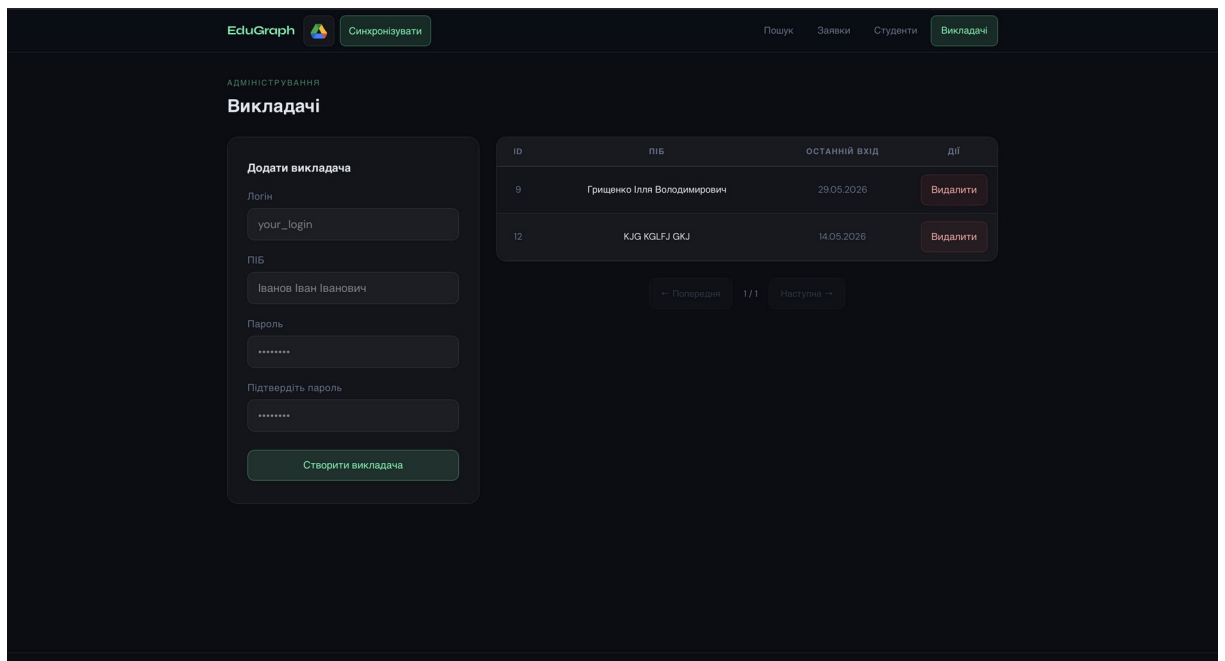


Рисунок А.7 – Сторінка керування викладачами

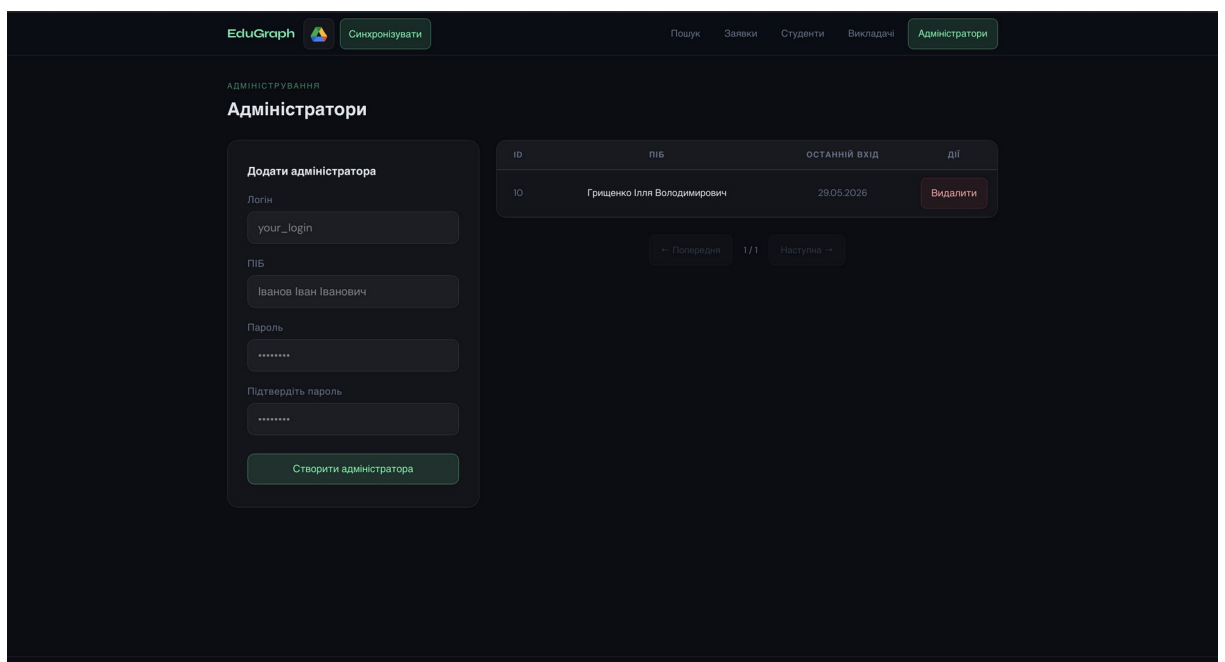


Рисунок А.8 – Сторінка керування адміністраторами

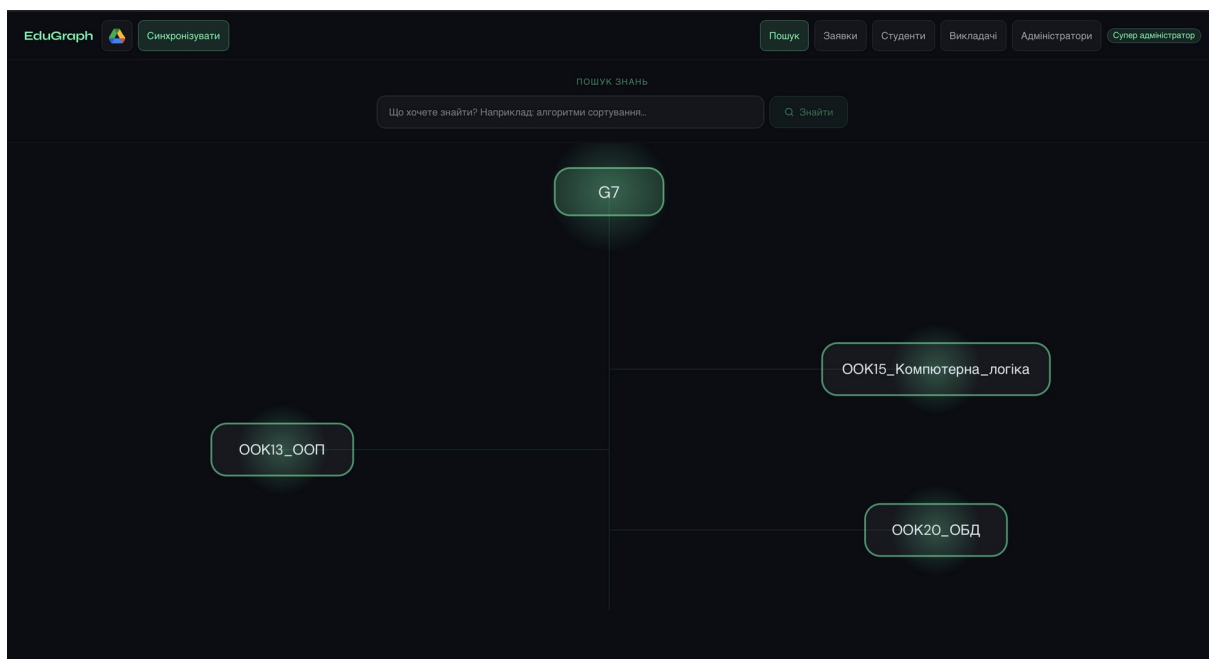


Рисунок А.9 – Навігація суперадміністратора в інтерфейсі пошуку

Додаток Б. Текстова специфікація API та моделей даних

Додаток Б подає текстову специфікацію API та моделей даних. Його призначення полягає в тому, щоб винести частину технічних деталей з основного тексту й залишити їх доступними для перевірки. Основна частина записки пояснює архітектуру та проєктні рішення, а додаток деталізує, які саме групи endpoint-ів і моделей використовуються в реалізації.

Специфікація не замінює OpenAPI-документацію, але є зручною для пояснювальної записки, оскільки описує endpoint-и людською мовою. Вона також допомагає показати зв'язок між frontend, backend і vector-search. Якщо в майбутньому API зміниться, цей додаток потрібно буде синхронізувати з фактичним openapi.json і кодом endpoint-ів.

У таблиці наведено не всі технічні деталі HTTP-відповідей, а основні групи дій, які є важливими для функціональної моделі. Для дипломної роботи цього достатньо, оскільки мета полягає не у повному копіюванні Swagger-опису, а в поясненні призначення API в архітектурі EduGraph.

Група	Маршрут / дія	Призначення
Auth	POST auth/login	Перевірка логіна й пароля, формування JWT з

		роллю користувача.
Auth	POST auth/signup	Створення заявки на реєстрацію без негайного створення користувача.
GoogleDrive	GET google-drive/folders	Повернення списку папок для побудови графу.
GoogleDrive	GET google-drive/root-folder-link	Повернення посилання на кореневу папку Google Drive.
GoogleDrive	GET google-drive/search-documents	Семантичний пошук документів через vector-search service.
GoogleDrive	POST google-drive/sync	Запуск синхронізації корпусу документів.
SignUpApplications	GET pending applications	Перегляд заявок, які очікують рішення.
SignUpApplications	POST approve / reject	Підтвердження або відхилення заявки.
Students / Teachers / Admins	GET / POST / DELETE	Керування користувачами відповідного типу.

Endpoint auth/login реалізує сценарій входу. Він приймає логін і пароль, виконує валідацію, шукає користувача, перевіряє пароль через Identity і повертає JWT. У цьому endpoint важливо, що backend не повертає зайвих даних користувача. Роль передається всередині token, а frontend читає її через auth utilities.

Endpoint auth/signup реалізує не створення користувача, а створення заявки. Це принципова відмінність від простих систем реєстрації. Заявка стає проміжною сутністю, яку має обробити уповноважена роль. Такий endpoint не повинен автоматично надавати доступ, інакше була б порушена модель контрольованого доступу до освітніх документів.

Endpoint `google-drive/folders` потрібний для графового інтерфейсу. Він повертає папки, які frontend перетворює на `folder`-вузли. Цей endpoint важливий навіть до пошуку, тому що базовий граф структури корпусу може бути побудований без результатів `semantic search`. Папки також використовуються для прив'язки знайдених документів за `folderName`.

Endpoint `google-drive/root-folder-link` повертає посилання на головну папку. Frontend кешує це посилання, щоб не виконувати зайвий запит кожного разу. Посилання потрібне для кнопки відкриття кореневого сховища. Це дає користувачу швидкий доступ до джерельної структури Google Drive.

Endpoint `google-drive/search-documents` є центральним для користувацького сценарію. Він приймає `Query` як параметр, перевіряє його, формує `VectorSearchRequest` і викликає `VectorSearchService`. Після отримання результатів backend мапить `VectorSearchResult` у власний `Response`. Це дозволяє відокремити внутрішній контракт FastAPI від frontend-контракту.

Endpoint `google-drive/sync` запускає синхронізацію. Він не повинен виконувати всю довгу роботу прямо в `request-response` сценарії без контролю, тому в системі передбачені фонові служби й черга примусового запуску. Для користувача це виглядає як дія «оновити корпус», але всередині запускається складний процес.

Endpoint-и `SignUpApplications` відповідають за адміністративний життєвий цикл заявок. `GetPendingApplications` повертає заявки, які очікують рішення. `Approve` створює користувача та змінює статус заявки. `Reject` змінює статус без створення користувача. Усі ці дії повинні бути захищені політиками ролей.

Endpoint-и `Students`, `Teachers` і `Admins` реалізують керування користувачами. Вони можуть мати схожу структуру, але різні політики доступу. Такий поділ робить API зрозумілим: кожна група endpoint-ів відповідає за певний тип користувача. Для frontend це також зручно, оскільки можна мати окремі api-модулі `studentsApi`, `teachersApi` і `adminsApi`.

Взаємодія frontend з API відбувається через `apiFetch`. Це означає, що endpoint-и не викликаються довільним `fetch` у кожній сторінці. Централізація запитів важлива для авторизації, обробки 401, налаштування base URL і єдиного підходу до JSON. У пояснювальній записці це можна подати як приклад зменшення дублювання на клієнтському рівні.

Взаємодія backend з FastAPI відбувається через `VectorSearchService`. Цей сервіс приховує HTTP-деталі й повертає `Result`. Якщо FastAPI недоступний або повертає помилку, backend отримує `ErrorDetails`. Це не усуває помилку, але робить її керованою та придатною для відображення або запису в статус індексації.

Модель	Ключові поля	Коментар
<code>SearchDocuments.Response</code>	Title, Url, FolderName, Content, Score, ChunkId, ChunkIndex	Контракт між backend і frontend для відображення результатів пошуку.
<code>VectorSearchRequest</code>	query, top_k, min_score, min_rerank_score	Контракт між ASP.NET Core API та FastAPI-сервісом.
<code>VectorDocumentDto</code>	id, title, content, url, content_hash, folder_name	Дані, які backend передає для індексації документа.
<code>IncomingDocument</code>	id, title, content, url, content_hash, folder_name	Pydantic-модель FastAPI для upsert.
<code>SearchResult</code>	document_id, chunk_id, chunk_index, title, content, url, score	Результат FastAPI після vector search і reranking.

`SearchDocuments.Response` є моделлю, яку frontend фактично використовує для побудови результатів. Title відображається як назва документа. Url використовується для переходу до Google Drive. FolderName дозволяє прив'язати результат до папки. Content показується в modal. Score,

ChunkId і ChunkIndex потрібні для технічної ідентифікації та пояснення результату.

VectorSearchRequest є моделлю запиту від backend до FastAPI. У простому випадку backend передає тільки Query, але FastAPI також підтримує top_k, min_score і min_rerank_score. Це означає, що в майбутньому backend може надати користувачу або адміністратору можливість налаштовувати кількість результатів і пороги релевантності.

VectorDocumentDto є моделлю документа для індексації. Вона передає id, title, content, url, contentHash і folderName. Ця модель не містить усіх полів UniversityDocument, а тільки ті, які потрібні vector-search сервісу. Це правильне рішення, оскільки Python-сервіс не повинен знати про всі доменні деталі backend.

IncomingDocument у FastAPI є Pydantic-відображенням документа. Для title і content встановлено обмеження max_length. Це захищає сервіс від надто великих або некоректних запитів. Хоча такі обмеження не замінюють повноцінну політику ресурсів, вони є першим рівнем контролю розміру вхідних даних.

SearchRequest містить query, top_k, min_score і min_rerank_score. Значення top_k обмежене, щоб користувач не міг запросити надто велику видачу. min_score дозволяє відсіювати слабкі vector matches. min_rerank_score дозволяє відкинути результати, які CrossEncoder оцінив надто низько.

SearchResult містить як vector score, так і rerank score. Це важливо для діагностики якості пошуку. Vector score показує близькість embeddings, а rerank score показує оцінку пари query-content. У frontend зараз основний акцент робиться на результаті й фрагменті, але ці оцінки можуть бути використані для майбутнього аналізу.

UniversityDocument є доменною моделлю, яка зв'язує Google Drive і vector index. GoogleDriveId використовується як стабільний зовнішній ідентифікатор. ContentHash показує, чи змінився текст. SearchIndexStatus

показує, чи відповідає vector index поточному тексту. SearchIndexError дозволяє не втрачати інформацію про проблеми індексації.

UniversityFolder потрібна для графу та збереження структури Google Drive. Вона має IsMain, що дозволяє відокремити кореневу папку від звичайних тематичних папок. Це важливо, бо root-вузол графу не є звичайною папкою з точки зору UI: він виконує роль центральної точки навігації.

User і SignUpApplication відображають організаційну частину системи. EduGraph не є анонімним пошуковиком, тому модель користувачів має бути пов'язана з ролями. Заявка дозволяє створювати користувачів контрольовано, а не через самостійну відкриту реєстрацію.

GraphNode і GraphLink на frontend не є backend-моделями, але вони важливі для інтерфейсу. GraphNode може представляти root, trunk, folder або document. GraphLink з'єднує вузли. Така внутрішня модель дозволяє перетворити список папок і список документів у графову структуру без зміни backend API.

Додаток В. Матриця ролей, вимог і сценаріїв перевірки

Додаток В деталізує рольову модель і сценарії перевірки. У головному тексті ролі описані концептуально, а тут вони представлені як матриця доступу. Такий формат зручний для перевірки відповідності між вимогами, frontend-маршрутами та backend-політиками.

Матриця не повинна сприйматися як остаточний юридичний регламент доступу. Вона описує технічну модель дипломного прототипу. Якщо система буде впроваджуватися в реальному університетському середовищі, ролі й права потрібно буде погодити з відповідальними особами.

Роль	Пошук	Заявки	Користувачі	Синхронізація	Особливості
Student	Так	Ні	Ні	Ні	Основний сценарій – пошук і перегляд результатів.

Teacher	Так	Так	Частково	За політикою	Може брати участь у керуванні навчальним доступом.
Admin	Так	Так	Так	Так	Основна адміністративна роль.
SuperAdmin	Так	Так	Так	Так	Додатково керує адміністраторами.

Роль Student має найвужчий набір прав. Вона потрібна для користувача, який шукає матеріали та переглядає результати. Студент не повинен керувати заявками, користувачами або синхронізацією, оскільки ці дії впливають на стан системи.

Роль Teacher має проміжний характер. У межах прототипу викладач може мати доступ до частини адміністративних сценаріїв, зокрема заявок або створення користувачів. Це відповідає ситуації, коли викладач бере участь у супроводі навчального доступу. Остаточний набір прав цієї ролі може бути уточнений перед впровадженням.

Роль Admin призначена для повного супроводу користувачів і корпусу документів. Адміністратор може працювати зі списками, заявками й синхронізацією. Ця роль повинна бути обмежена, оскільки неправильні дії адміністратора можуть вплинути на доступ користувачів або актуальність індексу.

Роль SuperAdmin відрізняється можливістю керувати адміністраторами. Це потрібно для відокремлення звичайного адміністрування від найвищого рівня керування системою. У невеликому прототипі така роль може виглядати надлишковою, але для реального середовища вона корисна.

Матриця доступу повинна перевірятися на двох рівнях. На backend рівні перевіряються policies. На frontend рівні перевіряються routes і навігація. Якщо frontend приховує сторінку, але backend дозволяє запит, це потенційна проблема. Якщо backend забороняє запит, але frontend показує сторінку, це UX-проблема. Обидва рівні мають бути узгоджені.

Рольова модель також впливає на тестування. Для кожної ролі потрібно перевірити дозволені й заборонені маршрути. Наприклад, Student має відкривати /student/search, але не /admin/admins. SuperAdmin має відкривати адміністративні сторінки. Teacher не повинен випадково отримати доступ до дій, які мають бути тільки для SuperAdmin.

ID	Сценарій	Передумова	Очікуваний результат
T-01	Успішний вхід	Користувач існує й пароль правильний.	Повертається JWT, відкривається маршрут ролі.
T-02	Невдалий вхід	Пароль неправильний.	Доступ не надається, показується помилка.
T-03	Подання заявки	Користувач заповнив форму.	Створюється pending-заявка.
T-04	Підтвердження заявки	Заявка очікує рішення.	Створюється користувач з роллю.
T-05	Синхронізація	Google Drive доступний.	Папки й документи оновлюються в SQLite.
T-06	Індексація	Є NotIndexed документи.	Створюються chunk-и у LanceDB.
T-07	Пошук	Індекс містить документи.	Повертаються ranked результати.
T-08	Граф	Frontend отримав folders і docs.	Будуються вузли й links, modal відкривається.

Сценарій T-01 перевіряє базову працездатність login. Він важливий, бо всі інші сценарії залежать від авторизації. Якщо token не формується або роль не читається, користувач не зможе правильно перейти до свого маршруту. Успішний результат означає, що backend, Identity, JWT і frontend auth utilities працюють узгоджено.

Сценарій T-02 перевіряє відмову при неправильних даних. Система не повинна уточнювати, що саме неправильне – login чи password, оскільки це може допомагати перебору. Достатньо повідомити, що дані неправильні. Також не повинно створюватися жодного токена.

Сценарії T-03 і T-04 перевіряють розділення заявки та користувача. Це одна з важливих бізнес-вимог. Якщо заявка одразу створює користувача, модель контрольованого доступу порушена. Якщо підтвердження заявки не створює користувача, сценарій реєстрації не завершується.

Сценарій T-05 перевіряє роботу із зовнішнім джерелом. Google Drive може містити папки, документи різних форматів і вкладені структури. Система повинна обійти дерево, отримати текст і посилання, а не тільки назви файлів. Також потрібно перевіряти видалення.

Сценарій T-06 перевіряє зв'язок SQLite і LanceDB. Якщо документ має статус NotIndexed, він повинен бути переданий у FastAPI. Після успішного upsert статус повинен змінитися на Indexed. Якщо статус залишається NotIndexed після успішного upsert, система буде повторювати індексацію.

Сценарій T-07 перевіряє semantic search. Успішним результатом є не просто HTTP 200, а наявність змістовних полів у відповіді. Для frontend потрібні title, url, content, folderName, chunkId і chunkIndex. Якщо будь-яке з ключових полів відсутнє, UI може працювати неправильно.

Сценарій T-08 перевіряє граф. Граф повинен містити root, папки й документи. Якщо документ має folderName, він повинен бути прив'язаний до відповідної папки. Якщо folderName відсутній, документ не повинен

втрачатися, а має бути прив'язаний до root. Modal повинен відкривати фрагмент саме того документа, по якому натиснув користувач.

Ці сценарії можна виконувати вручну під час демонстрації або поступово автоматизувати. Для backend доцільні integration tests, для frontend – browser/e2e перевірки, для vector-search – HTTP tests з невеликим контрольним набором документів. У межах поточної записки вони фіксують план перевірки й показують, що тестування не обмежується візуальним оглядом інтерфейсу.

Додаток Г. Розгорнута інструкція та чек-лист супроводу

Додаток Г містить розгорнуту інструкцію та чек-лист супроводу системи. Основна інструкція користувача в розділі 3.9 описує роботу коротко, щоб не перевантажувати головний текст. У цьому додатку ті самі процеси деталізовано з погляду користувача, адміністратора та особи, яка супроводжує систему після розгортання.

Додаток корисний з двох причин. По-перше, він показує, що система EduGraph розглядається не тільки як програмний код, а як інструмент, яким мають користуватися реальні ролі. По-друге, він фіксує експлуатаційні обмеження, які можуть бути неочевидними під час демонстрації: якість документів, доступність Google Drive, стан індексації та ресурси Python-сервісу.

Цей додаток доповнює основну інструкцію користувача й показує практичний порядок використання системи після розгортання. Він потрібний для того, щоб відокремити короткий опис у розділі 3.9 від розгорнутих експлуатаційних кроків і чек-листів супроводу.

Крок	Дія користувача	Реакція системи
1	Відкрити сторінку входу.	Показується форма логіна й пароля.
2	Увести облікові дані.	Backend перевіряє користувача й повертає JWT.
3	Перейти до пошуку.	Frontend відкриває

		маршрут відповідно до ролі.
4	Увести змістовий запит.	API передає запит до vector-search сервісу.
5	Натиснути на документ у графі.	Відкривається modal з релевантним фрагментом.
6	Відкрити оригінал.	Браузер переходить до Google Drive документа.

Користувач студентського типу починає роботу зі сторінки входу. Якщо він не має облікового запису, він не повинен звертатися безпосередньо до адміністратора поза системою, а може подати заявку. Такий підхід централізує процес отримання доступу та залишає слід у вигляді заявки. Для навчального середовища це зручніше, ніж ручне створення облікових записів без історії.

Після входу студент бачить пошуковий інтерфейс. Його не потрібно навчати технічним поняттям embeddings, vector score або reranking. Інтерфейс має бути сформульований навколо простої дії: ввести запит і переглянути результати. Якщо система працює правильно, студент бачить не тільки документ, а й фрагмент, який пояснює релевантність.

Під час формулювання запиту користувачу варто рекомендувати писати змістовні фрази, а не тільки одне слово. Семантичний пошук краще використовує контекст, якщо запит містить предмет, дію або характеристику. Наприклад, запит «вимоги до оформлення звіту з практики» є інформативнішим, ніж просто «звіт».

Якщо результатів немає, це не завжди означає, що документ відсутній. Можливо, запит сформульовано занадто вузько, документ ще не проіндексовано або текст не був витягнутий з файла. Користувачу можна рекомендувати змінити формулювання, використати ширші поняття або звернутися до адміністратора, якщо очікуваний документ точно має бути в корпусі.

Коли користувач відкриває modal документа, він має прочитати фрагмент і вирішити, чи потрібно відкривати повний документ. Система не повинна заохочувати використання фрагмента як повної заміни джерела. Фрагмент є навігаційною підказкою, а не остаточним навчальним матеріалом.

Перехід до Google Drive потрібний для перевірки повного контексту. У документі можуть бути таблиці, рисунки, примітки або попередні розділи, які не потрапили в chunk. Тому у навчальному середовищі коректним сценарієм є «знайти через EduGraph – перевірити в оригіналі – використати матеріал».

Викладач або адміністратор працює з системою ширше. Окрім пошуку, він може переглядати заявки. Під час підтвердження заявки потрібно перевіряти, чи справді користувач належить до відповідної групи або має право на роль. Технічна кнопка approve не замінює організаційне рішення.

Під час створення користувачів адміністратор має бути уважним до ролі. Неправильно видана роль може відкрити зайві сторінки або, навпаки, заблокувати потрібний сценарій. Особливо обережно потрібно працювати з роллю SuperAdmin, оскільки вона має найширші права.

Запуск синхронізації слід виконувати після помітних змін у Google Drive. Якщо додано один документ, можна дочекатися фонового процесу. Якщо потрібно швидко перевірити новий матеріал, ручний запуск прискорює оновлення. Водночас не варто запускати синхронізацію багато разів поспіль без потреби, оскільки це створює навантаження на Google Drive API та vector-search сервіс.

Користувачі повинні розуміти, що результат пошуку залежить від індексації. Якщо документ щойно додано, він може з'явитися в Google Drive, але ще не бути в LanceDB. Тому між додаванням документа та його появою в semantic search може бути затримка. Це нормальна властивість системи з окремим індексом.

Зона супроводу	Що контролювати	Ознака проблеми
Google Drive	Структуру папок, доступ service account,	Папки не оновлюються або документи

	формати файлів.	не читаються.
SQLite	Наявність користувачів, заявок, документів, статусів індексації.	Документи не переходять у Indexed або мають Failed.
FastAPI	Старт моделей, доступність /health, розмір vector_db.	Пошук повертає 500 або порожній результат для очевидних запитів.
Frontend	Маршрути, токен, граф, modal, safe URL.	Користувач бачить неправильну сторінку або не відкриває документ.

Супровід Google Drive починається зі структури папок. Папки повинні мати зрозумілі назви, тому що вони відображаються у графі. Якщо папка названа технічно або випадково, користувач побачить цю назву в інтерфейсі. Тому порядок у Google Drive прямо впливає на якість графової навігації.

Доступ service account має бути стабільним. Якщо обліковий запис, через який backend читає Google Drive, втратить права, синхронізація перестане працювати. У такому випадку frontend може продовжувати показувати старі результати, але корпус не буде оновлюватися. Це потрібно контролювати через логи синхронізації та статуси документів.

Супровід SQLite включає контроль цілісності таблиць і резервне копіювання. Навіть якщо Google Drive є джерелом документів, SQLite містить користувачів, заявки, статуси індексації та локальний стан корпусу. Втрата цієї БД означатиме втрату облікових записів і необхідність повторної синхронізації.

Супровід vector_db включає контроль розміру, відповідності документам і можливість повторної побудови. У поточному прототипі vector_db можна розглядати як похідне сховище, яке за потреби можна відтворити з SQLite і Google Drive. Проте для великого корпусу повторна

індексація може тривати довго, тому резервне копіювання також може бути доцільним.

FastAPI-сервіс має власні ресурсні вимоги. Моделі SentenceTransformer і CrossEncoder завантажуються в пам'ять, а embeddings і reranking потребують обчислень. Якщо сервер має мало пам'яті або CPU, пошук може бути повільним. Перед production-розгортанням потрібно визначити мінімальні ресурси для стабільної роботи.

Логи є важливою частиною супроводу. Backend повинен фіксувати помилки Google Drive, vector-search і індексації. Python-сервіс фіксує vector result і rerank result, що корисно для діагностики. У майбутньому можна додати централізований збір логів через OpenTelemetry або інший observability stack.

Чек-лист перед фінальним поданням роботи має включати перевірку структури записки, актуальності схем, відповідності назв розділів методичним вимогам, оновлення переліку посилань, перевірку української мови, перегляд рисунків і таблиць, а також підтвердження того, що описані технології відповідають фактичному коду.

Окремо потрібно перевірити, що тема з LLM пояснена коректно. Оскільки поточна реалізація не генерує відповіді чат-моделлю, у тексті має бути чітко зазначено, що використовується LLM-орієнтований retrieval-підхід на основі transformer-моделей embeddings і CrossEncoder reranking. Це усуває можливе протиріччя між назвою теми та реалізацією.

Перед демонстрацією потрібно підготувати контрольний сценарій: запустити backend, FastAPI і frontend; переконатися, що /health працює; виконати login; завантажити папки; виконати пошук; відкрити modal; перейти до Google Drive. Цей сценарій повинен бути коротким і стабільним, щоб під час захисту не залежати від випадкового запиту.

Після захисту система може розвиватися як внутрішній сервіс. Для цього потрібно поступово додати автоматизовані тести, production deployment, резервне копіювання, фільтри, метрики якості пошуку та, за потреби,

генеративний LLM-модуль. Поточна архітектура дозволяє робити ці кроки послідовно.

Додаток Д. Глосарій термінів і технічних понять

Глосарій потрібний для уніфікації термінів у пояснювальній записці. Проєкт EduGraph поєднує веб-розробку, бази даних, Google Drive інтеграцію та методи обробки природної мови. Через це одні й ті самі поняття можуть тлумачитися по-різному. У додатку подано пояснення саме в контексті цієї роботи, а не універсальні академічні визначення.

Особливу увагу потрібно приділити термінам LLM, semantic search, embeddings і reranking. У темі роботи використано формулювання «на основі LLM», але фактична реалізація зосереджена на retrieval-підході. Тому в записці важливо не створювати враження, що система вже генерує відповіді чат-моделлю. Вона використовує transformer-моделі для пошуку та ранжування фрагментів.

Термін	Пояснення в контексті EduGraph
Semantic search	Пошук, який порівнює зміст запиту й документа через векторні представлення, а не тільки точні слова.
Embedding	Числовий вектор, що представляє зміст текстового фрагмента або запиту.
Chunk	Фрагмент документа, який індексується окремо для точнішого пошуку.
Reranking	Другий етап ранжування, де CrossEncoder уточнює порядок кандидатів.
Vector store	Сховище embeddings, у проєкті представлено LanceDB.
JWT	Токен, що підтверджує автентифікацію користувача й містить роль.
VSA	Vertical Slice Architecture, організація backend за функціональними зрізами.

Google Drive sync	Процес оновлення локального корпусу на основі папок і документів Google Drive.
-------------------	--

Термін «LLM-орієнтований підхід» у цій роботі означає використання сучасних мовних transformer-моделей для представлення та порівняння тексту. Це ширше, ніж класичний keyword search, але вужче, ніж повноцінна генеративна RAG-система. EduGraph знаходить джерельні фрагменти та показує їх користувачеві, а не створює нову відповідь від імені моделі.

Термін «semantic search» означає пошук за змістом. Якщо користувач вводить запит, система не обмежується перевіркою входження слів у документ. Вона перетворює запит і фрагменти документів у embeddings і порівнює їх у векторному просторі. Завдяки цьому система може знаходити фрагменти, які близькі за змістом, навіть якщо формулювання не збігається повністю.

Embedding є числовим представленням тексту. Для людини embedding нечитабельний, але для алгоритму він дозволяє вимірювати близькість між запитом і фрагментом. У EduGraph embeddings створюються моделлю `intfloat/multilingual-e5-base`. Для документів використовується префікс `passage`, а для запиту – `query`, що відповідає логіці моделі.

Chunk є частиною документа. Поділ на chunk-и потрібний тому, що освітні документи можуть бути великими й містити багато тем. Якщо індексувати весь документ як один вектор, релевантний фрагмент може загубитися серед нерелевантного тексту. Chunking дозволяє знайти конкретний змістовий уривок.

Overlap між chunk-ами потрібний для збереження контексту на межах фрагментів. Якщо текст розрізаний рівно по межі chunk-a, важлива частина речення може опинитися в сусідньому фрагменті. Overlap 150 символів зменшує цей ризик і робить embeddings стабільнішими для пограничних випадків.

Vector store є сховищем embeddings. У EduGraph цю роль виконує LanceDB. Воно зберігає vector разом із метаданими chunk-a: document_id, title, content, url, folder_name і content_hash. Завдяки цьому після пошуку можна повернути не тільки число релевантності, а й змістовний результат для користувача.

Cosine similarity є способом оцінити близькість векторів. Якщо вектори нормалізовані, cosine distance можна перетворити на score. У FastAPI-сервісі після пошуку в LanceDB score обчислюється як 1.0 мінус distance. Далі результати нижче min_score відкидаються.

Reranking є другим етапом пошуку. Перший етап швидко знаходить кандидатів через vector similarity, але він може помилятися в порядку. CrossEncoder отримує пару query і content та оцінює її точніше. Це повільніше, але застосовується тільки до обмеженого набору кандидатів, тому залишається практичним.

JWT є токеном, який підтверджує вхід користувача. У EduGraph token містить роль, яку frontend використовує для маршрутизації, а backend – для політик доступу. Якщо token недійсний або прострочений, користувач повинен повернутися на сторінку входу.

Vertical Slice Architecture означає організацію backend не навколо великих шарів controllers/services/models для всього застосунку, а навколо окремих функцій. Кожен endpoint має власні типи й handler. Для EduGraph це зручно, бо функції auth, заявки, користувачі та Google Drive мають різні правила.

Result pattern означає, що метод може повернути успішний результат або ErrorDetails без використання винятку як звичайного способу керування потоком. Це робить бізнес-помилки явними. Наприклад, неправильний пароль або невалідний документ є очікуваними ситуаціями, які можна повернути як Result failure.

Google Drive sync означає не просте копіювання файлів, а процес синхронізації стану. Система читає папки, документи, текст, link і hash,

порівнює з локальною БД, створює нові записи, оновлює змінені й видаляє ті, що зникли. Після цього змінені документи індексуються у vector store.

Graph navigation означає подання результатів як вузлів і зв'язків. У EduGraph root-вузол відповідає головному сховищу, folder-вузли – папкам, а document-вузли – знайденим документам. Така модель допомагає бачити не тільки релевантність, а й місце документа в структурі корпусу.

Modal document preview означає перегляд фрагмента без переходу з пошукового інтерфейсу. Це важливо, бо користувач може швидко оцінити, чи справді результат корисний. Якщо фрагмент підходить, користувач відкриває повний документ у Google Drive.

ContentHash є способом визначити зміну документа. Якщо текст змінився, hash також змінюється, і документ потрібно переіндексувати. Це дозволяє не будувати vector index заново для всього корпусу при кожній синхронізації.

SearchIndexStatus показує стан документа щодо vector index. NotIndexed означає, що документ потребує індексації. Indexed означає, що він успішно доданий до індексу. Failed означає, що індексація завершилася помилкою, яку потрібно проаналізувати.

Таким чином, глосарій допомагає однаково використовувати терміни в усій записці. Це особливо важливо для міждисциплінарної теми, де поєднуються програмна інженерія, веб-архітектура та методи семантичного пошуку.